Designing Graphic User Interfaces for Cormas

A Cormas tutorial by Pierre Bommel & Christophe Le Page, 2022

This document presents the concepts and the way to generate graphical user interfaces (GUI) in Smalltalk VW. It is adapted to Cormas in order to create GUI for your model. It is divided in three parts:

1. the use of the Dialog class to quickly generate simple interfaces,
2. the creation and installation of specific interfaces with the addition of components,
3. Some examples of complete interfaces.

Note that this document summarizes the main elements needed to design GUIs for Cormas. This document is available on Cormas web site: https://cormas.cirad.fr/en/logiciel/didacticiel as Tutorial 5. A more complete doc provides a full description of GUI: the official VW document GUIDevGuide.pdf that is available in vw7.6nc/doc/ of your VW folder, or on the Cormas website: Learning how to use Cormas.

The code of the main examples of this tutorial are available in the "InteractiveModel" model. This toy model is present by default in the Cormas package and can also be downloaded on this page: http://cormas.cirad.fr/en/applica/interactiveModel.html.

1- Dialog: using predefined simple GUIs **4**
Warn the user 4
Get confirmation 4
Request an information 4
Choose an option 4
Request files 5
Cormas dialogs 6
2- Designing a GUI with UI Painter **7**
The MVC principle 7
Available examples of simple GUIs 8
Using the Cormas Toolbox 8
Creating a new empty interface 9
Adding basic widgets 12
Using the information of the interface 22
Opening the GUI from your model 23
Closing the GUI and setting up your model 24
3- GUI for initializing an ABM **25**
Adding more elaborate widgets 26
Opening an interface when initializing your model 32
SubCanvas: reuse the previous canvas 34
4- GUI for one agent **38**
Open the GUI for one agent 38
Close GUI and change the agent's attributes 39
5- Advanced settings **39**
Adapting the widgets when the GUI size is changed 39
Listening to events 40
6- Examples of ABM interfaces **43**

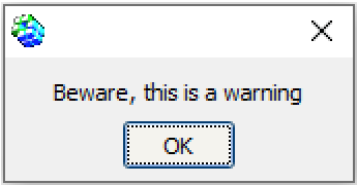
Displaying parameters: prices and costs 43
Assigning a name and a color to the players 45
Specializing the built-in interfaces of Cormas 46
Spatial grid as Game board 47
Adapting the charts 48
Adapting the parameters' interface 49
Adapting the Scenario builder interface (SimManager) 49
Adapting Cormas' interface 49
Appendix **51**
Loading VW examples of GUI 51
Simple dialog vs ApplicationModel 52
Load or capture an image 53
Adding the attribute 'model' to the GUI 54
Setting GUI with the previous versions of Cormas 55

1- Dialog: using predefined simple GUIs

The class *'Dialog'* provides a variety of simple user interfaces. At class level, *Dialog* offers useful methods such as: `#warn:`, `#confirm:`, `#request:`, etc.
Each interface created with *Dialog* is pre-emptive: it holds the control until it is closed. In other words, the execution flow is stopped and the user cannot click on another interface while the Dialog window is open (see further explanation in Simple dialog vs ApplicationModel).

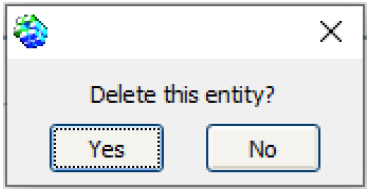
Warn the user

Code to open a *'warning'* interface: `Dialog warn: 'Beware, this is a warning'.`



Get confirmation

Code to open a *'confirmation'* interface: `Dialog confirm: 'Delete this entity?'`



The return value of this instruction is a ***Boolean***, so you can write something like:

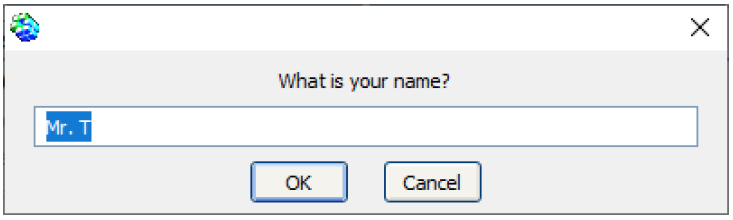
```
(Dialog confirm: 'Delete this entity?') ifTrue: [...] ifFalse: [...].
```

(Note: A click on close [x] returns `false`).

Request an information

Code to open a *'request'* interface:

```
Dialog request: 'What is your name?' initialAnswer: 'Mr. T'
```



The result of this interaction will be a ***String***. If 'Cancel' or close [x] have been selected, it returns an empty string.

Choose an option

Here are some examples of choosing methods (see *Dialog* → 'multiple choice dialogs').

Choosing from buttons

```
Dialog
	choose: 'Are you tired yet?'
	labels: (Array with: 'absolutely' with: 'sort of' with: 'not really')
```

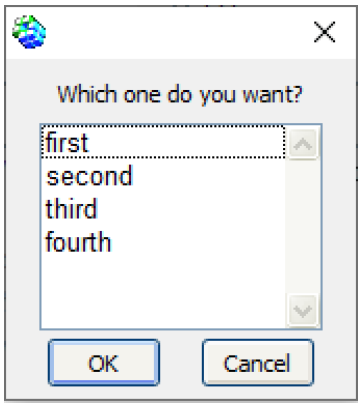
```
values: #(#yes #maybe #no)
default: #maybe
```



In that case, the result of this interaction will be a ***Symbol***: `#yes, #maybe` or `#no` (but you can decide the type of the results with the parameters of this instruction).
(Note: A click on close [x] does nothing).

Choosing from a list

```
Dialog
        choose: 'Which one do you want?'
        fromList: #('first' 'second' 'third' 'fourth')
        values: #(1 2 3 4)
        lines: 8
        cancel: [#noChoice]
```

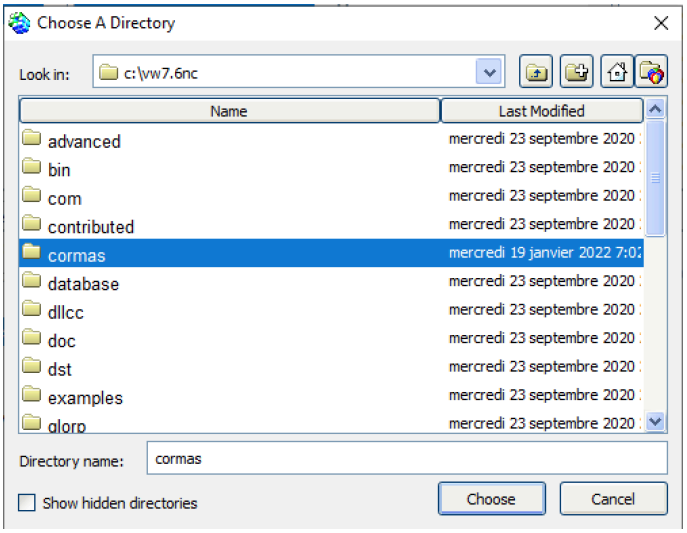


The result of this interaction will be a ***number*** (1, 2, 3 or 4) or a ***Symbol*** (`#noChoice`), but again, you can decide the type of the results with the parameters of this instruction.

Request files

Request a directory

```
Dialog requestDirectoryName: 'Choose a Directory:' default: '*'
```

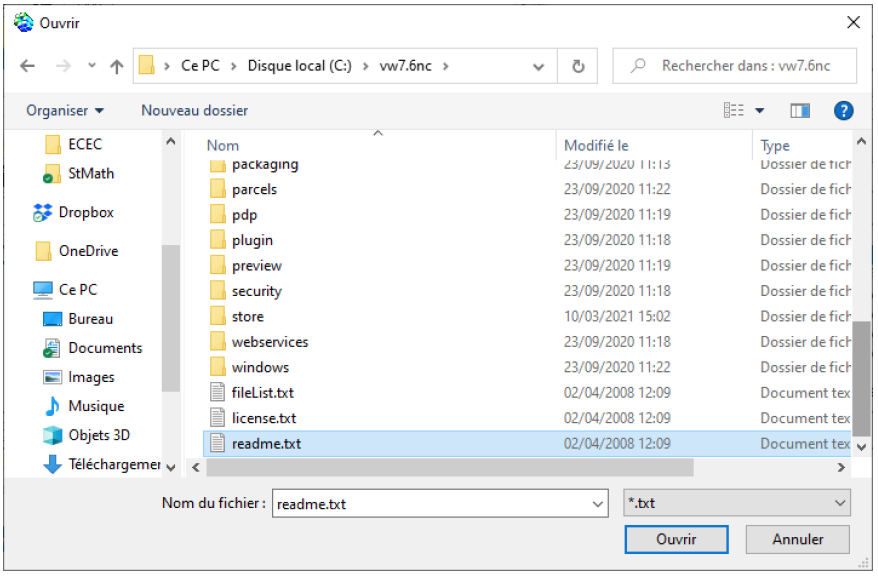


The result of this interaction will be a ***String***, like: `'C:\vw7.6nc\cormas'`.
Then this string must be transformed as Filename, such as in the following example:

```
directory := Dialog requestDirectoryName: 'Choose a Directory to save the
text files:' default: (Cormas modelPath: self cormasModel class name).
directory = '' ifTrue: [^nil].
directory := directory asFilename.
```

Request a file

```
Dialog requestFileName: 'Name a file' default: '*.txt'
```



This will return a ***String***, such that: `'C:\vw7.6nc\readme.txt'`

Cormas dialogs

Just notifying…

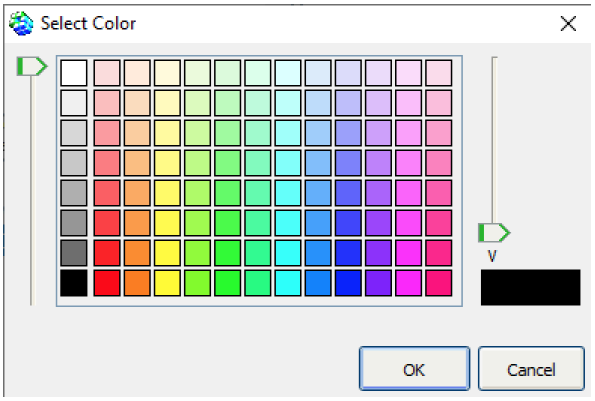
```
Cormas justNotify: 'Player A turn' forMilliseconds: 500
```



In that case, the execution flow is stopped for 500 milliseconds displaying a window with no button. This prevents the user from clicking.

Color chooser

```
Cormas colorChooser
```

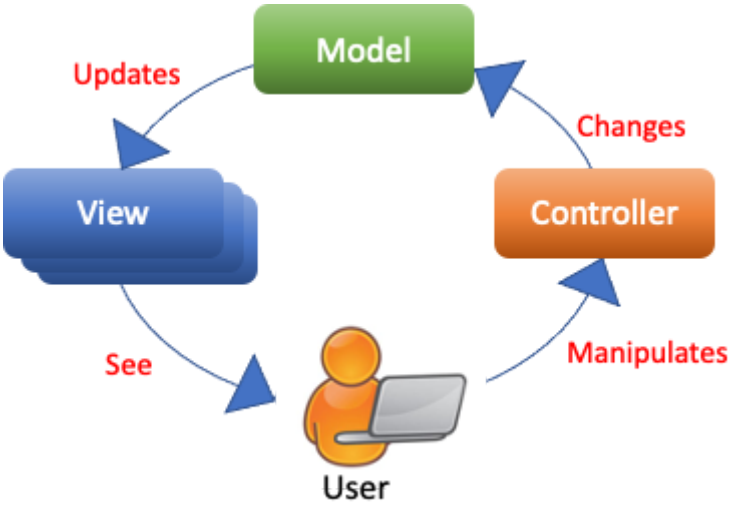


This will return a ***ColorValue*** (or nil) such as `ColorValue red: 0.375046 green: 1.0 blue: 1.0` or `ColorValue yellow`.

2- Designing a GUI with UI Painter

The MVC principle

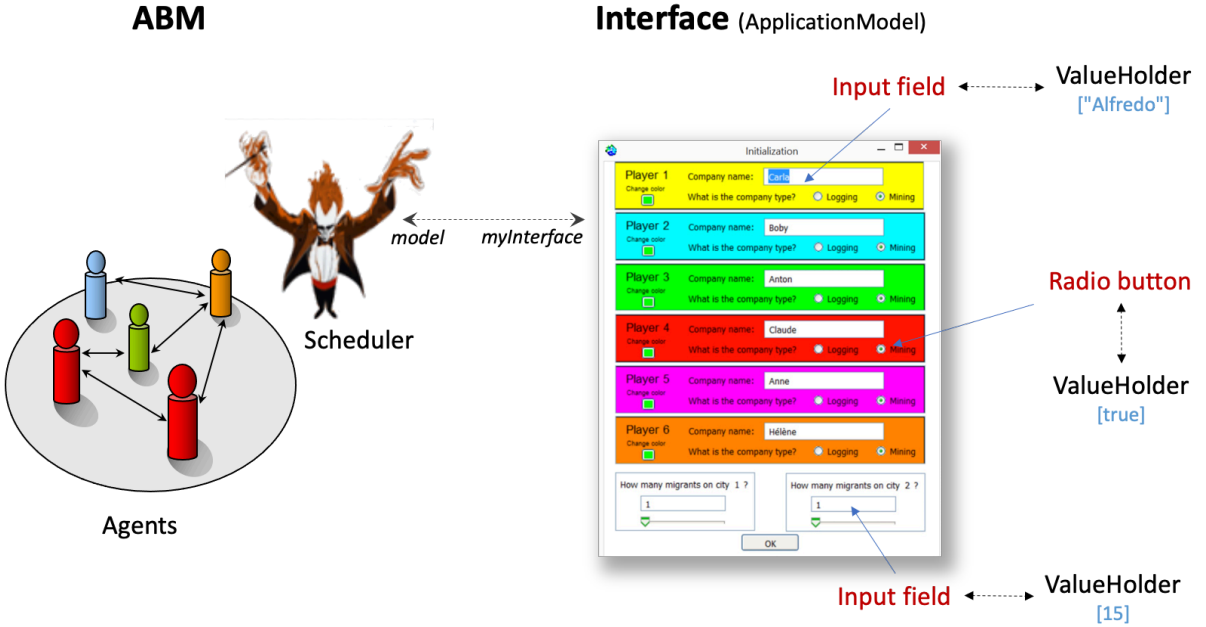
The philosophy of Cormas (as well as of Smalltalk) is to separate the Model from its View(s) and the way to Control it. MVC (abbreviation of Model - View - Controller[1]) is the design pattern commonly used for developing applications. It means that there is no direct interaction between the users and the model:



[1] MVC was proposed by Trygve Reenskaug for Smalltalk-79 in the 1970s.

Here, the 'Model' does not only mean your full ABM, but any element of your model like for example the 'energy' attribute of an agent. According to the MVC approach, this attribute (that is an object) is also a model that can be visible as a specific color of the agent displayed on the spatial grid and that can be modified by using manipulation interfaces.

And the same principle applies to a user interface. Mainly, your new interface will be a subclass of `ApplicationModel`, an abstract class that uses *UIBuilders* to generate the user *Window*. So, when designing a new interface (for managing a simulation or just one agent), you need to create a new class that inherits from *ApplicationModel*. Thus, your interface may be viewed as an intermediary between the view and controller combination and your ABM:



But this interface is also a model (an instance of *ApplicationModel*) composed of widgets: Input fields, radio buttons, lists, checkboxes, … Each widget is associated with a **ValueHolder** which contains data ("Alfredo", 15, true,…) specified at initialization and that can be changed by the user.

Available examples of simple GUIs

Most of the following examples are available in the "`InteractiveModel`" toy model. This model is present by default in Cormas package and can also be downloaded on this page: http://cormas.cirad.fr/en/applica/interactiveModel.html

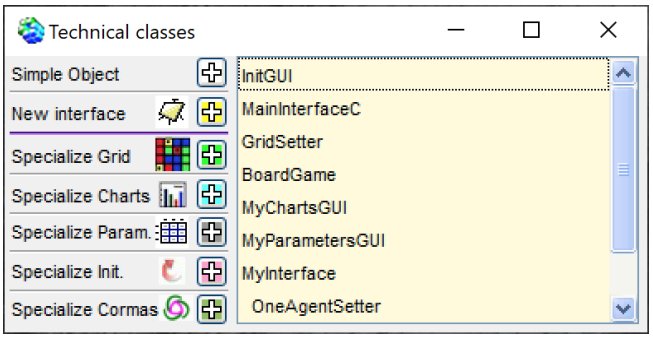
Furthermore, VW offers a set of examples of simple GUI. They can be very useful to understand how a widget works. The section Loading VW examples of GUI in the Appendix explains how to load an example and to use it.

Using the Cormas Toolbox

Cormas (v. 2022.3) offers a set of tools for technical classes to facilitate the design and installation of new interfaces. This toolbox is available from the menu `Tools` → `Toolbox` or from the `[>]` button on the *'entities'* interface:



Thus, the toolbox opens:



By clicking on the various [+] buttons, you can create new classes such as:

- **SimpleObject** that isn't a subclass of entity of your model but rather a technical class to instantiate for example the lines of a DataSet.
- **New interface** to set up, for example, the players at the beginning of an interactive simulation.
- **New grid** that specializes the `SpatialInterface` of Cormas in order to design the game board.
- **New charts** interface that specializes the probes Interface of Cormas in order to adapt it to your model.
- **New Parameter** interface that specializes the `ParameterInterface` of Cormas.
- **New Initialize interface** that specializes `SimManager` class in order to design new scenario builder interfaces when initializing a simulation[2].
- **New Cormas** interface that specializes the main interface of Cormas.

The toolbox avoids the quite tedious configuration of these technical classes so that they are saved with the model (i.e. they are automatically integrated into the Namespace and the Package of your model).
For those who wish to use an older version of Cormas, the procedures described in the Appendix should be applied.
The specialization of the spatial grid, the charts, the parameters, the Initialize interfaces and Cormas is explained in the section Specializing the Cormas interfaces.

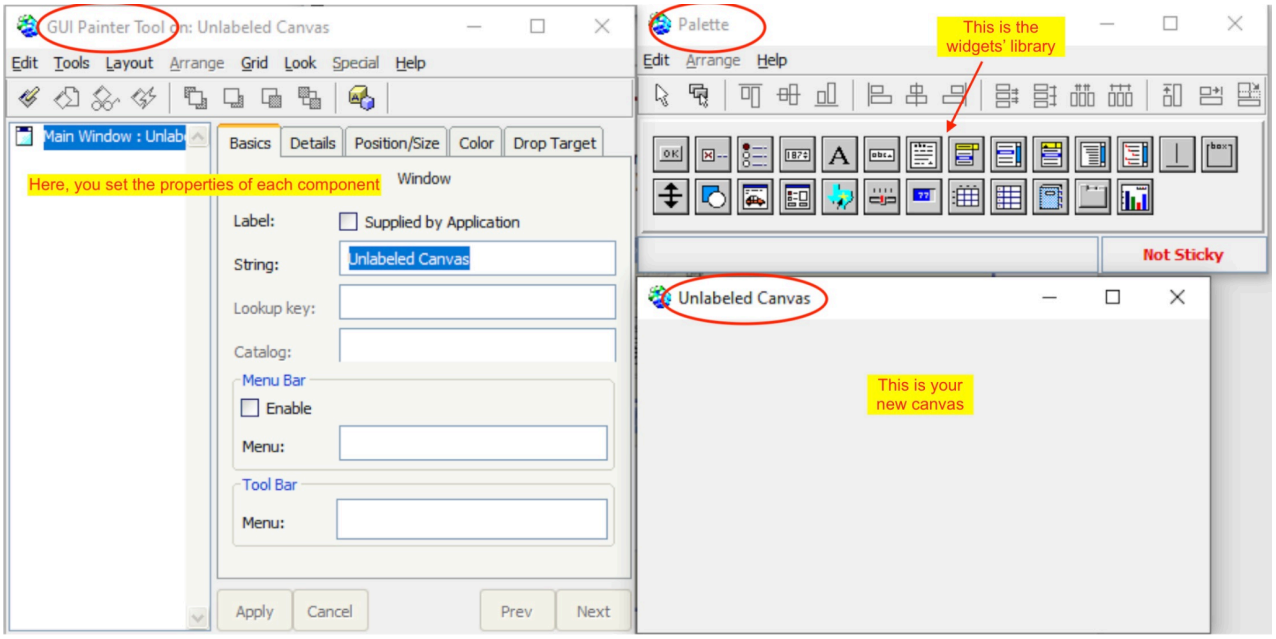
Creating a new empty interface

Building a GUI is done by visually selecting, placing, and arranging widgets on a **Canvas** (this process is also called "painting"). The Canvas is where you place and arrange the widgets of your GUI.
To open a new canvas (and the UI Painter), click on the **'New interface' button** of the Cormas toolbox:  on the toolbox. After giving a name to this new class (ie. "`MyInterface`"), 3 windows will open up:

1. An *'Unlabeled Canvas'* (lower right), that is the new interface on which you will place widgets,
2. A *Palette* (top right), that presents the widgets to put on your canvas,

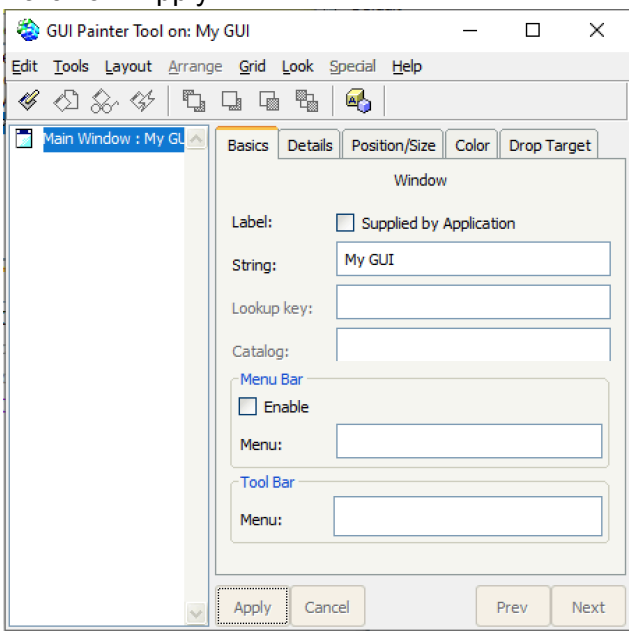
[2] As `CormasModel` is a subclass of `ApplicationModel`, it could be possible to create GUI from the main class of your model, but we do not recommend to do so in order to differentiate the elements of your model

3. A *GUI Painter Tool* (left) to set the properties of each widget.

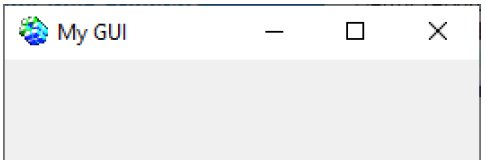


Rename the interface

To rename your interface, change the String field into the property setter interface (called *GUI Painter Tool*), then click on “Apply”:

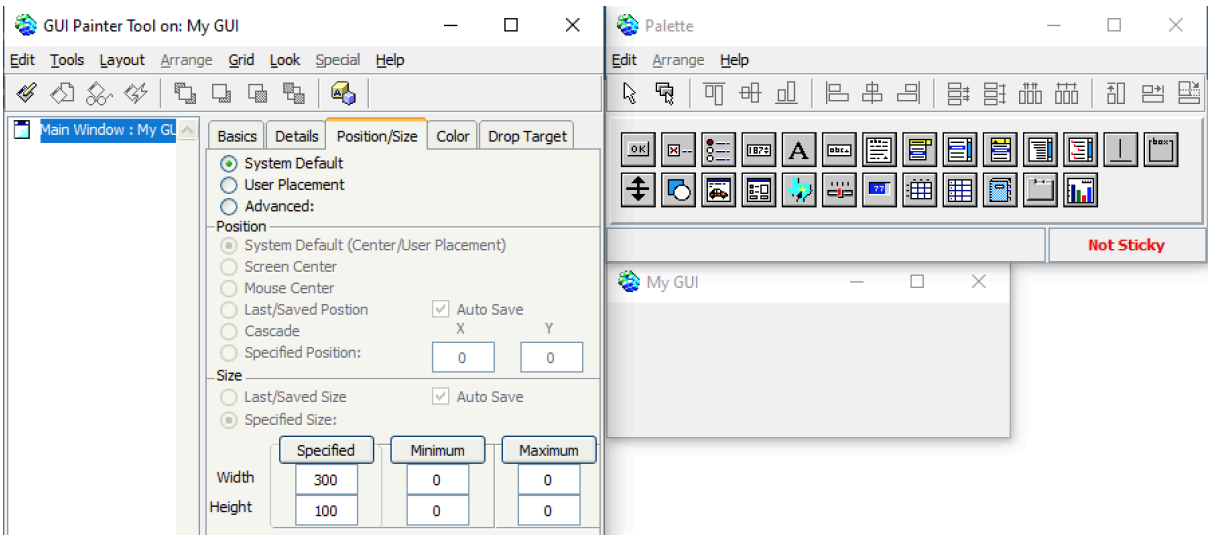


Now the title of your interface has the correct name:



Set the size of the interface

Select the 3rd tab (Position/Size) of the *GUI Painter Tool*:

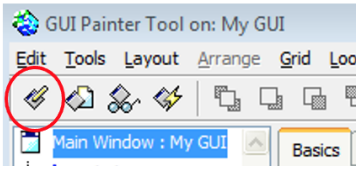


Modify the size of your canvas (with the mouse), then click on the “Specified” button. If you want a fixed size interface (that cannot be changed by user), you must give the same ‘Width’ and ‘Height’ values to the *‘Specified’*, *‘Minimum’* and *‘Maximum’* fields.
If the user is allowed to change this size, then you have to define the ‘Minimum’ (and ‘Maximum’) ‘Width’ and ‘Height’ values. If it is 0, then there is no size constraint.

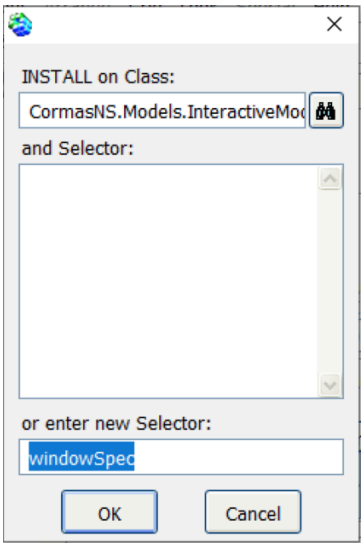
Save your GUI

Now that you have a minimalist interface, let’s save it, that is to say to “install” it in your model. This writes the canvas specification into a “*spec*” method. This specification will be used when opening the window (you can also edit the specification later by reopening it in the UI Painter).

To install your GUI, click on the first button of the *GUI Painter Tool*  :



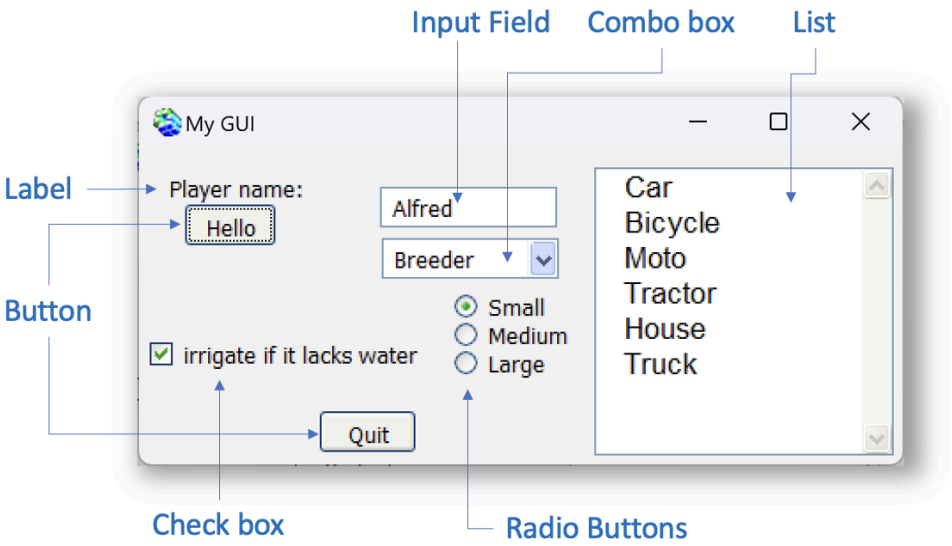
By default, the specification will be stored into your class (`MyInterface`) with a specification method called *`windowSpec`* by default:



Each time a new widget will be added on the interface, the ‘install’ procedure must be performed in order to save those changes.

Adding basic widgets

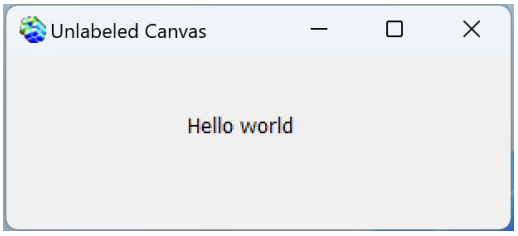
Widgets are elements (like labels and buttons) that can be included on your GUI. To add a component to your canvas, you simply click on the corresponding icon in the *Palette* to select it, and then click in the canvas to place the widget. Here are the main widgets you will need for developing your GUI: *Label, Button, Checkbox, InputField, ComboBox, List, RadioButtons*.



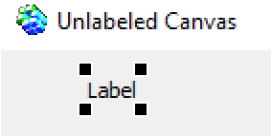
In the following, these widgets (and the way to characterize them) are presented in order of increasing difficulty:

Label

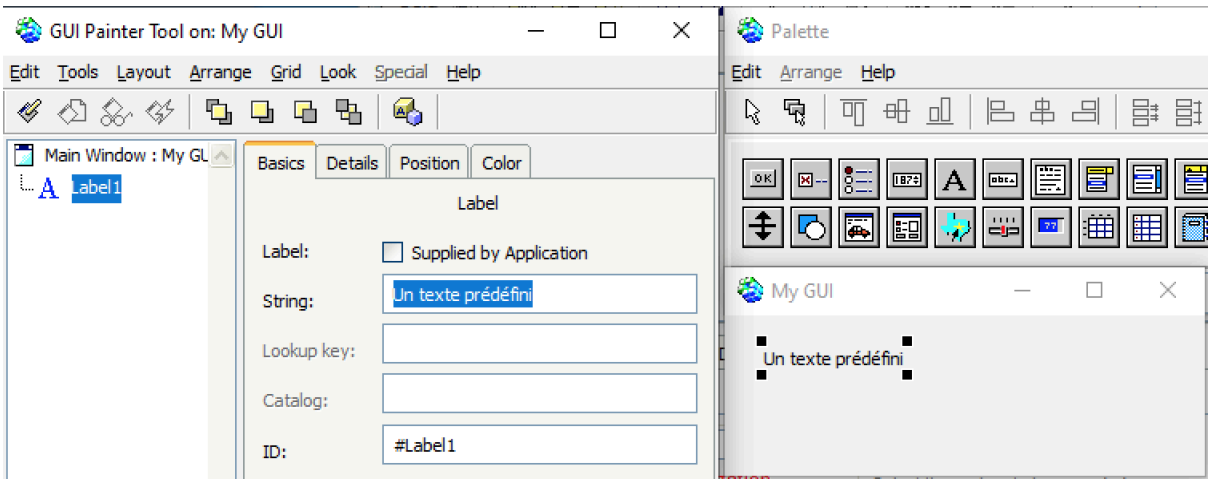
Example:



To add a simple text to your canvas, click on the Label icon  in the *Palette*, then click on the canvas to place it:



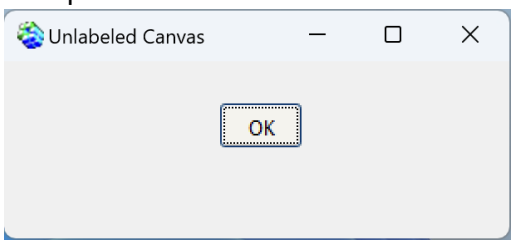
To set the value of this label, change the String field into the *GUI Painter Tool*, then click on "Apply":



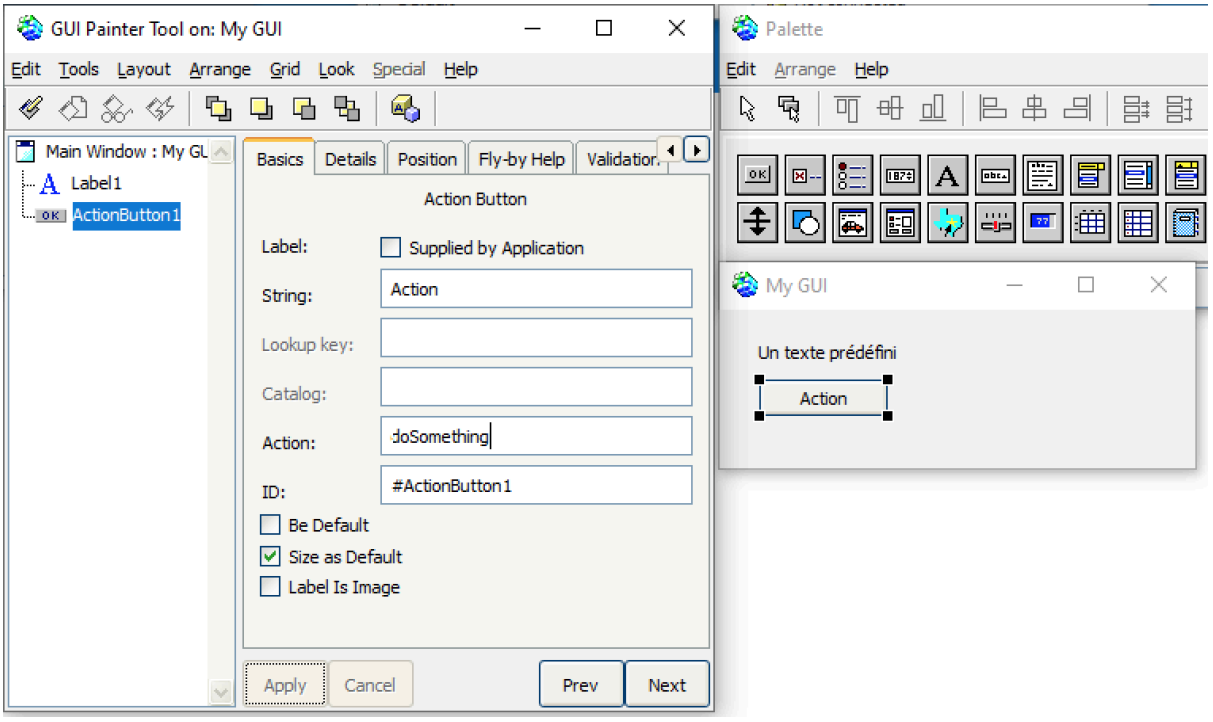
Note that this widget can also be an image (see Add image). If you want to modify the text when the interface is open, you should use an InputField instead.

Button

Example :



To add a button to your canvas, click on the Button icon  in the *Palette*, then click on the canvas to place it:

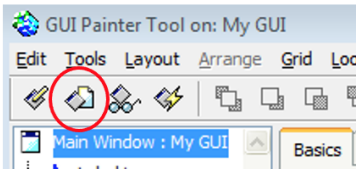


Rename this button (Ex: ‘Hello’) by changing the field String of the *GUI Tool* (then ‘Apply’).

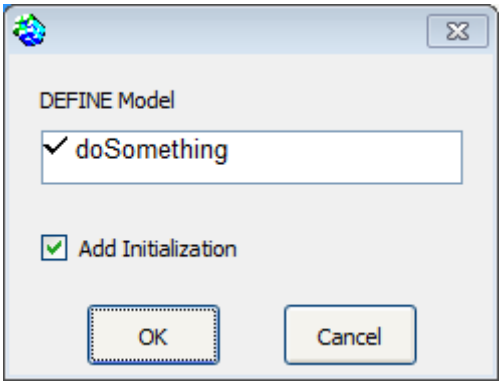
Specify the method name that will be performed in the field 'Action' (then 'Apply'). Example, write **doSomething**. This means that when the user will click on this button, the method `doSomething` will be executed.

Installing the button

The method `doSomething` should be available at the 'instance' level of the class of your interface. An empty method will automatically be defined when clicking on the 'Define' button  of the GUI Tool.



Just click on "OK" to generate this method:



Defining the behavior of the button

For the moment, the method associated to this button is just declared, but it is empty:

```
doSomething
    "This stub method was generated by UIDefiner"
    ^self
```

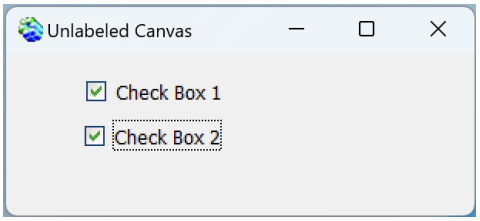
It must be changed in order to do something (for example, for opening a warning):

```
doSomething
    Dialog warn: 'Hello Alfred'
```

The behavior of a button can be more complexe. The section Close the GUI… explains how to close and use the results of an interaction with a GUI.

Checkbox

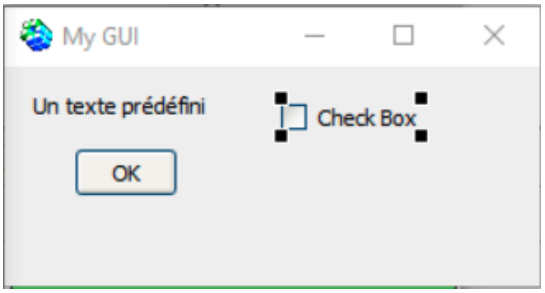
Example:



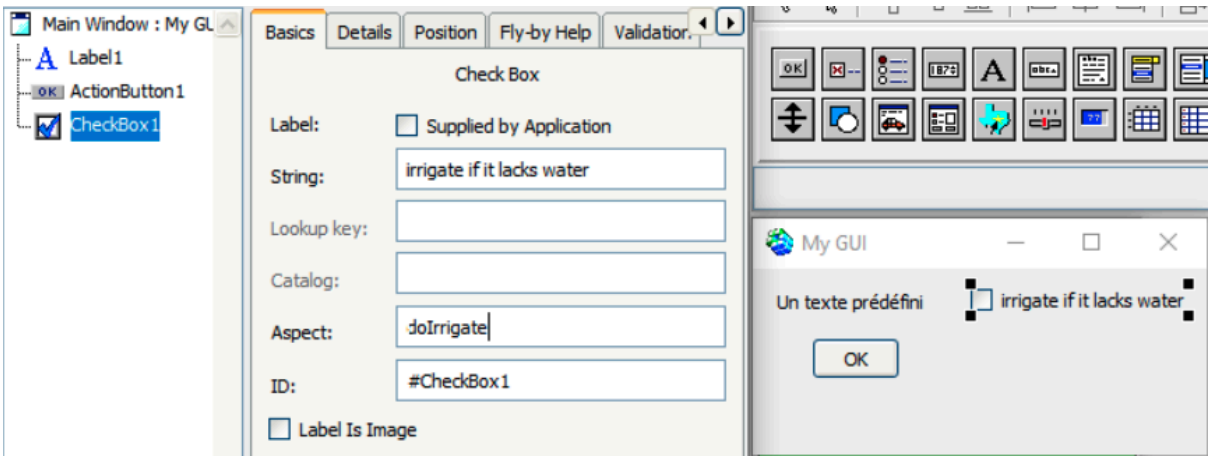
A checkbox allows the user to turn on or turn off an attribute (so, the underlying model is a *Boolean*). Selecting one check box has no effect on the others, so users can select as many

check boxes as they want. If you want only one attribute to be selected at a time, then use *Radio buttons* instead.

To add a checkbox to your canvas, click on the Checkbox icon  in the *Palette* (2nd button), then click on the canvas to place it:



Change the name of the checkbox in the String field of the GUI Tool (then 'Apply'). Ex: "`irrigate if it lacks water`".
Enter an Aspect property for the checkbox (ex: **`doIrrigate`**). The attribute **`doIrrigate`** will be a Value holder for this checkbox that will contain *true* when the check box is selected and *false* when it is not selected.

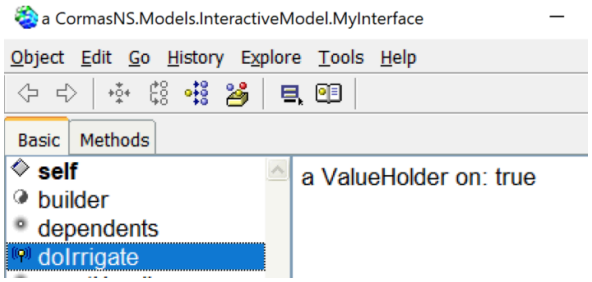


Apply the properties and install the canvas (first button of the *GUI Painter Tool* ), then use the 'Define' button  of the GUI Tool to create the attribute (called **`doIrrigate`**) and its getter accessor method. This accessor answers a ValueHolder on a boolean. Edit the `doIrrigate` method to set the default value of the attribute: a value holder containing *true* if you want the checkbox to be selected by default and *false* otherwise. Example for checkbox selected by default:

```
doIrrigate

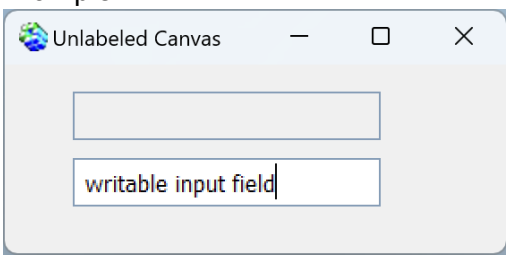
    doIrrigate ifNil: [doIrrigate := true asValue].
```

Now, while inspecting an instance of `MyInterface`, you can see that the attribute **`doIrrigate`** is a `ValueHolder on: true`.



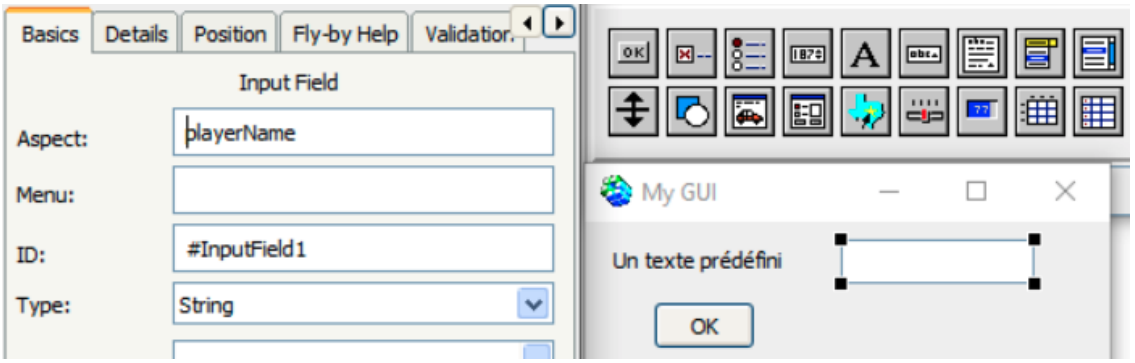
InputField

Example:



An input field is used for both entering[3] and displaying data. You can also use a field in read-only mode (like a Label) when you just want to display data.

1. From the *Palette*, add an input field to the canvas (6th button: ).
2. On the basics property page (ie. *GUI Tool*), in the field's Aspect, enter the name of the getter method for the field's value model (ex: **`playerName`**).

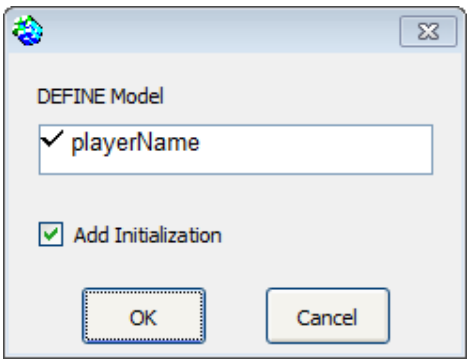


An input field displays the content of a valueHolder (that holds data). When the input field accepts input from a user (Control), it sends this data to the valueHolder for storage; when the field needs to update its View, it asks its value model for the data to be displayed.

Installing the inputField

The attribute **`playerName`** should be defined at the 'instance' level of your interface. It also needs a getter accessor called `playerName`. Both attribute and accessor are automatically defined when clicking on the 'Define' button  of the *GUI Tool*.

[3] When a field has a restricted list of valid entries, consider using a Combo box instead.



Here is the code generated by the *Define* button:

```
playerName
	"This method was generated by UIDefiner.  Any edits made here
	may be lost whenever methods are automatically defined. The
	initialization provided below may have been preempted by an
	initialize method."
	^playerName isNil
		ifTrue:
			[playerName := String new asValue]
		ifFalse:
			[playerName]
```

So this new attribute is an instance of ValueHolder with an empty String by default.

Set a default value

To set another default name, you can either:

1. Change the accessor `playerName` as following

```
playerName
	^playerName ifNil:[playerName := 'Alfred' asValue]
```

or

2. Add an `initialize` method:

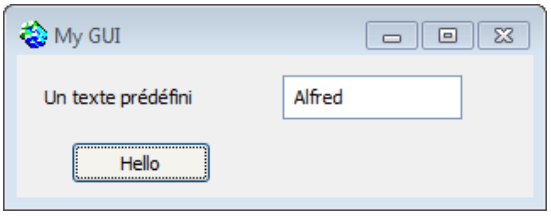
```
initialize
	playerName := 'Alfred' asValue.
```

or

```
	self playerName value: 'Alfred'.
```

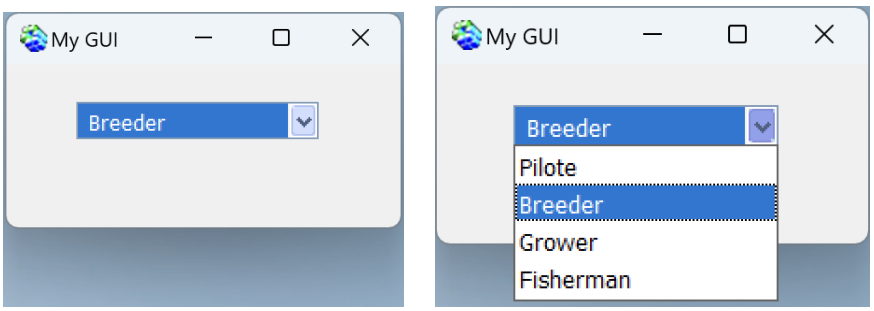
Defining the behavior of a modification of the InputField

To instantiate the interface, click on the 4th button  :



Combo Box

Example:



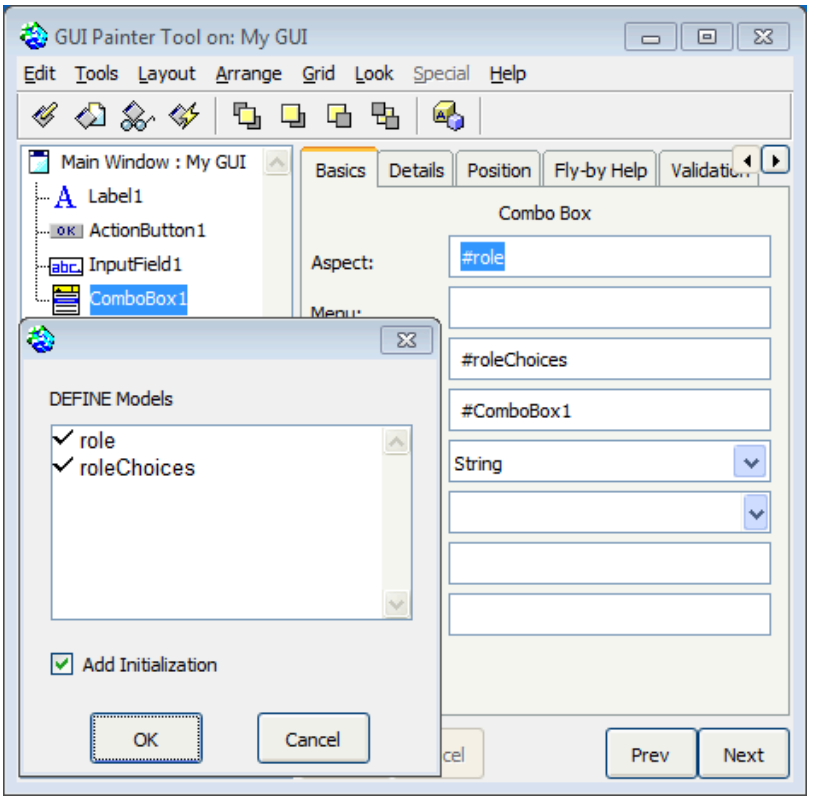
As the user can write anything in an InputField, a Combo box will restrict him/her to choose an element from a given list. To create a Combo box:

1. From the *Palette*, add a Combo box to your canvas (10th button: ).
2. On the basics property page (ie. *GUI Tool*),
   a. in the field's Choices, enter the name (ex: **`roleChoices`**) of the method that returns the list,
   b. in the field's Aspect, enter the name (ex: **`role`**) of the attribute that will be the default value selected in the list.

Installing the combo box

For a ComboBox, two attributes (**`roleChoices`** and **`role`**) should be defined at the 'instance' level of your interface. They must be accessible through two getter accessors called `roleChoices` and `role`.

The attributes and their accessors are automatically defined when clicking on the 'Define' button  of the *GUI Tool*:



The getter accessor of the attributes **`roleChoices`** looks like:

```
roleChoices
```

```
"This method was generated by UIDefiner.  Any edits made here
may be lost whenever methods are automatically defined. The
initialization provided below may have been preempted by an
initialize method."
^roleChoices isNil
	ifTrue:
		[roleChoices := List new asValue]
	ifFalse:
		[roleChoices]
```

So, by default, the attribute **`roleChoices`** is an instance of ValueHolder of an empty List. By default, the attribute **`role`** is a ValueHolder on an empty String.

Set the default values

You can change the default value for each attribute, either in the getter accessors, or in the `initialize` method:

1. From the accessors
   - Change the accessor **`role`** as following

```
role
^role ifNil:[role :=  'Grower' asValue]
```

   - Change the accessor **`roleChoices`** as following

```
roleChoices
	| list |
	^roleChoices
		ifNil:
			[list := List new.
			list  add: 'Grower';
				add: 'Breeder';
				add: 'Fisherman';
				add: 'Pilote'.
			roleChoices:= list asValue]
```

or

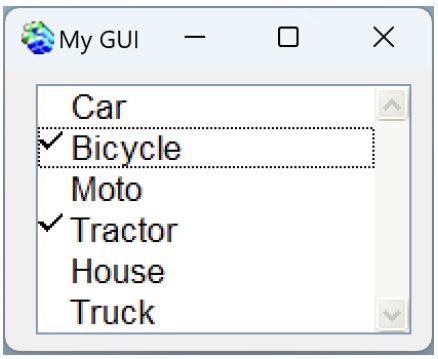
2. From the `initialize` method:

```
initialize
	| listRoles |
 self playerName value: 'Alfred'.
 role := 'Grower' asValue.
 listRoles := List new.
 listRoles add: 'Grower';
          add: 'Breeder';
          add: 'Fisherman';
          add: 'Pilote'.
 roleChoices:= listRoles asValue.
```

After having saved your interface (see Save your GUI), you can test it by clicking on the 4th button  .

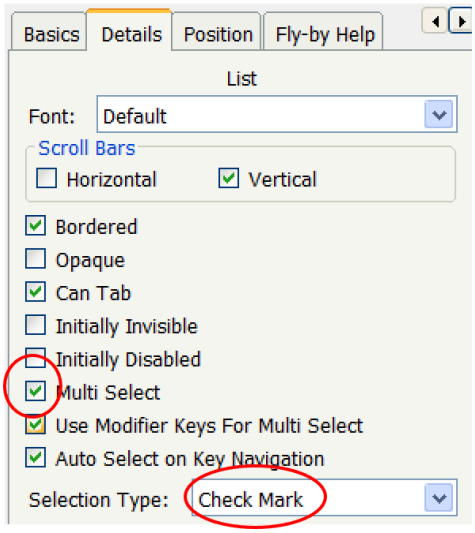
List

Example:



Like a Combo box, a list allows the user to choose an element (or several) from a given list. To create a List:

1. From the *Palette*, add a List to your canvas (9th button: ).
2. On the basics property page (ie. *GUI Tool*),
    a. in the field's Aspect, enter the name (ex: **`goodSelections`**) of the attribute that will be an instance of list (or rather, a SelectionInList that holds a ValueModel with a list).
    b. If the user is enabled to select more than one item, you have to opt for a `MultiSelectionInList`. To do so, go to the `'Details'` tab and check "Multi Select". You can also specify which type of selection to choose :



Installing the list

Only one attribute (**`goodSelections`**) should be defined at the 'instance' level of your interface. It is accessible through a getter accessors called `goodSelections` that is automatically defined when clicking on the 'Define' button  of the *GUI Tool*.

The getter accessor of the attributes **`goodSelections`** looks like:

```
goodSelections
	^goodSelections isNil
		ifTrue:
			[goodSelections := MultiSelectionInList new]
		ifFalse:
			[goodSelections]
```

(if only one selection is authorized, the default value will be `SelectionInList`)

Set the default values of the list

You can change the default value for the attribute, either in the getter accessors, or in the `initialize` method. Example from the **`initialize`** method:

```
initialize
      | listRoles |
  playerName := 'Alfred' asValue.
 role := 'Farmer' asValue.
listRoles := List new.
listRoles add: 'Farmer'; add: 'Breeder'; add: 'Fisherman'; add: 'Pilote'.
roleChoices:= listRoles asValue.

 self goodSelections list: listRoles.
```

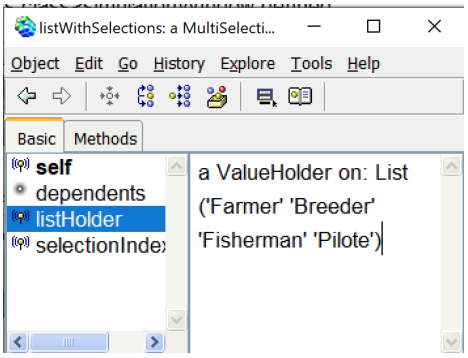
Here, the ComboBox and the List store the same list: `listRoles`. But of course they can be different, like `'Bicycle' 'Moto' 'House' 'Tractor' 'Truck' 'Car'`:

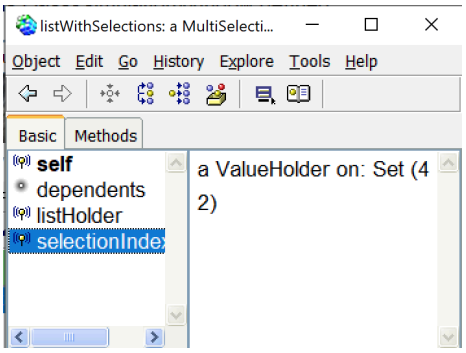
```
self goodSelections list: (Good allTerminalClasses collect: [:cl |cl name asString]).
```

After having saved your interface (see Save your GUI), you can test it by clicking on the 4th button  .



Inspecting the **`goodSelections`** attribute shows that it is an instance of `MultiSelectionInList` with 3 attributes, one of which is a valueHolder that contains the list with the elements as proposed in the initialization, and the other is also a valueHolder with the indexes of the 2 selected elements of this list:

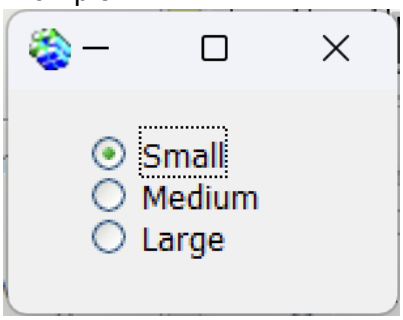




This information will be used in the ABM when closing the interface (see further Closing the GUI and setting up your model).
If you want the list to adapt when the user expands the interface, you have to change the position of this widget. See the explanations in Adapting the widgets when the GUI size is changed.

Radio buttons

Example:

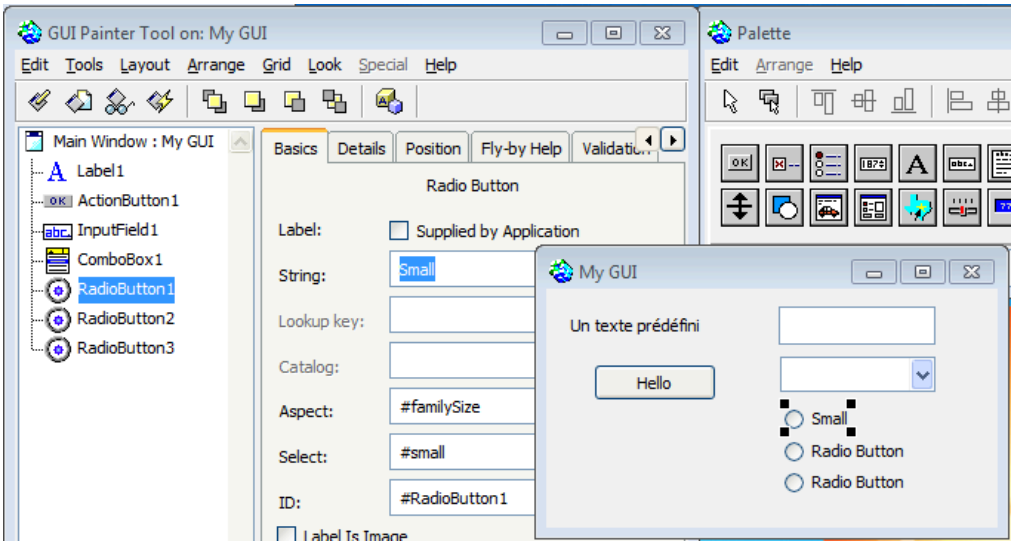


Radio buttons allow the user to select one option among several: this is an exclusive selection (if you want to enable multiple choices, use Checkbox instead). Selecting a radio button causes any other button in its group to be deselected. So the **group** is an important concept of radio buttons: it allows a user to select one option from a limited list of choices.
Ex: here we want to create 3 radio buttons ("Small", "Medium", "Large") to allow the user to choose the size of a family (the group). This means that **`familySize`** must be an attribute of the interface and each radio button set a value for this attribute. To create 3 Radio buttons:

1. From the *Palette*, add 3 Radio buttons to your canvas (3rd button: ).
2. On the basics property page (ie. *GUI Tool*),
    a. in the field's String, enter the name of each radio button that will be display after the button (ex: `Small`, Medium, or Large)
    b. in the field's Select, enter the value of the selected radio button (ex: **`#small`**). It corresponds to the value of the attribute.
    c. in the field's Aspect, enter the name of the **group**, ie. the attribute of the interface (ex: **`familySize`**).

Installing the radio buttons

The attribute **`familySize`** (and its accessor) should be available at 'instance' level of the current class of your interface. It is automatically defined when clicking on the 'Define' button  of the *GUI Tool*:



Set a default value

You can change the default values for **`familySize`**, either by its getter accessor, or in the `initialize` method:

1. From the accessor
    - Change the accessor **`familySize`** as following

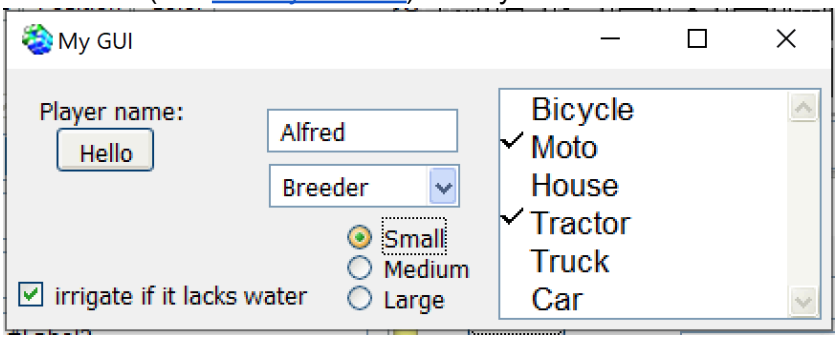
```
familySize
^familySize ifNil:[familySize :=  #small asValue]
```

or

2. From the `initialize` method:

```
initialize
        | listRoles |
playerName := 'Alfred' asValue.
role := 'Farmer' asValue.
listRoles := List new.
listRoles add: 'Farmer';
           add: 'Breeder';
           add: 'Fisherman';
           add: 'Pilote'.
roleChoices:= listRoles asValue.
self goodSelections list: (Good allTerminalClasses collect: [:cl |cl name asString]).
familySize :=  #small asValue  "or: self familySize value:  #small"
```

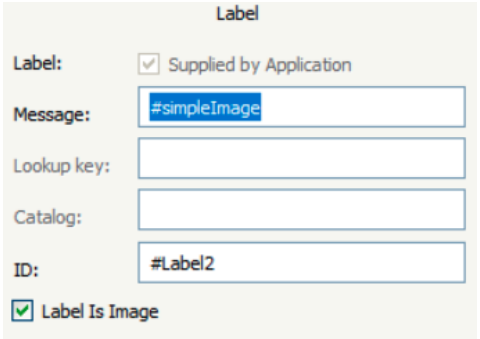
Then save your interface (see Save your GUI). Now your interface looks like:



Each time the user click on a radio button, a corresponding symbol (#small, #medium or #large) is holded by the attribute **familySize**.

Add image

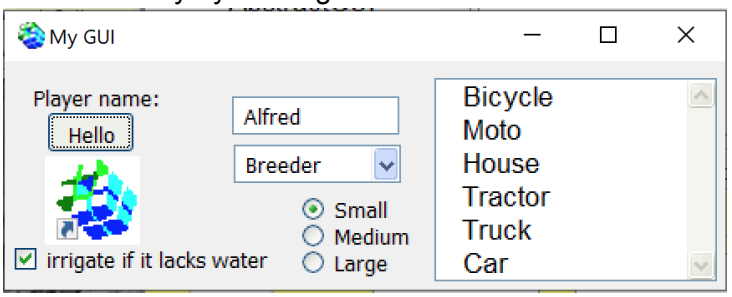
To add an image on your interface, just add a new Label, but put it as image: "*Label is Image*", and in the field Message, write the name of the image:



In this case, `simpleImage` must be a method defined at class level of your interface that answers an image. To load or capture an image, see the Load or capture an image section.

A modification of `simpleImage` when the interface is open doesn't change the image displayed on the GUI. If you need to dynamically change the image, you should rather use a more elaborate widget such as the ViewHolder  .

Now, save (see Save your GUI) this simple interface. As it doesn't need extra information to initialize it, you can test it directly by clicking on the 4th button  of the *GUI Tool*:

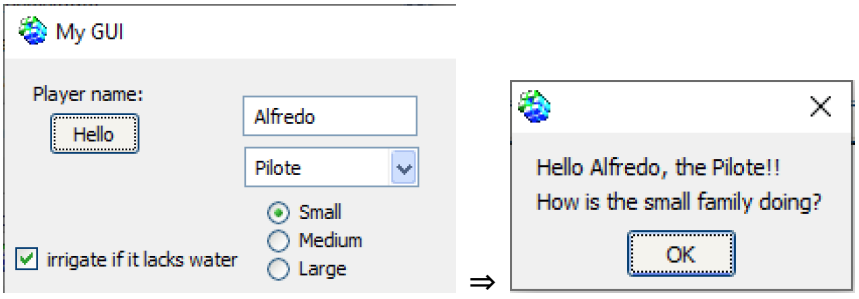


Using the information of the interface

In the Button section, the 'Hello' button has been defined with a fixed behavior: opening a Dialog with 'Hello Alfred'.
Instead of using a frozen text, you can reuse the information of the various fields of the interface, such as **playerName**, **role** and **familySize**. For that, let's change the behavior of the button.
First, change the 'Action' field to `#sayHello` (then save and define the widget).
Then, modify the `sayHello` method as following:

```
sayHello
	| aString |
	aString := 'Hello ', self playerName value, ', the ', self role value,'!!'.
	aString :=   aString, '
How is the ', self familySize value, ' family doing?'.
	Dialog warn: aString
```

Here, the `sayHello` method uses 3 attributes of the GUI to set the string to be displayed: `playerName`, `role` and `familySize`. But as they are instances of valueHolders, the message `value` is sent to each of them to get the stored data (for example: `self playerName value`).
Then each time you click on the 'Hello' button, the warning is adapted to the current interface state:

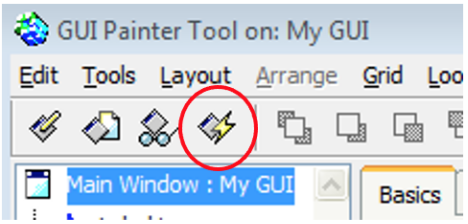


The data stored by the interface can also be used by your model when closing the GUI (see Performing Final Actions).

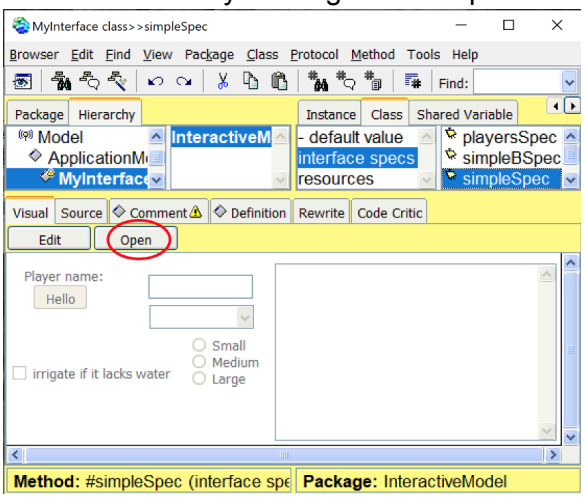
Opening the GUI from your model

Simple opening

As the current interface is simple and does not require any particular initialization that requires external data, it can be tested by opening it directly by clicking on the 4th button  of the GUI Tool:



It can also be opened from the browser by clicking on the 'Open' button:



Opening the GUI from your model

The usual way of opening an interface is to instantiate the Interface and to ask it to open. For example, from the `init` method of your own model, just write:

```
myInterface := MyInterface new. "#initialize is automatically sent within #new"
"myInterface aSpecificInitialization."  "Here a specific init can be called"
myInterface open.
```

If your interface specification has a different name than `#windowSpec`, then, instead of `open`, send `openInterface:` (`open` is equivalent to `openInterface: #windowSpec`).

```
myInterface openInterface: #windowSpec.
```

Setting the Startup Location of the interface

Once an interface is displayed, it is quite difficult to modify its content. It is therefore better to display it at the last moment, after having set everything as you want (`allButOpenInterface:`). Then, you can access its Builder or its window to specify the position:

1 Build the interface up to the point of opening the window.
2 Get the window from the interface builder.
3 Ask the window to open itself within a specified rectangle, using screen coordinates, in pixels.
If you want to open your interface at the point 50@50 of your screen (0@0 is the top - right point), write:

```
openMyInterface
 | bldr win |
 myInterface := MyInterface new.
 bldr := myInterface allButOpenInterface: #simpleSpec.
 win := bldr window.
 win openIn: (50@50 extent: win minimumSize).
```

This way of doing is useful, especially if you need to catch the builder and modify the interface while it is already open (see 5- Advanced settings).

Closing the GUI and setting up your model

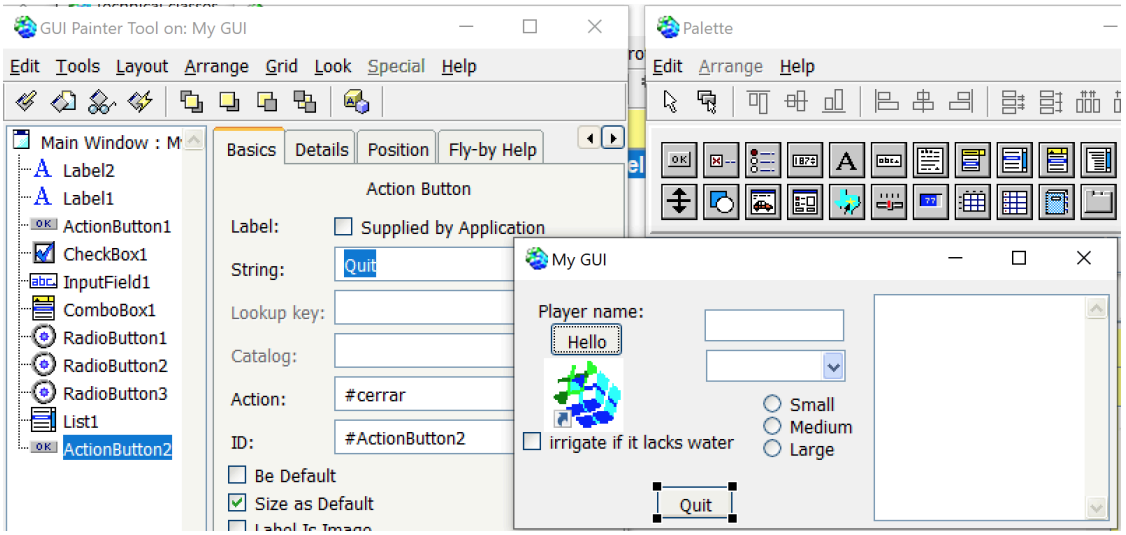
Simply close

The `ApplicationModel` class (super class of `MyInterface`) provides a means of closing a window, such as clicking on the close icon `[X]`. Often it is desirable for an application to provide a `close` method in its interface, such as a 'Quit' button. An interface might also be closed as a side effect of some conclusive user action.
When an application model is running one or more windows, you can close it (or all of them at once) by sending `closeRequest` to the application.
For example, we can ask the interface to close after 3 second:

```
initSimpleGUI
      | myInterface |
      myInterface := MyInterface new.
      myInterface openInterface: #simpleSpec.
      (Delay forSeconds: 2) wait.
      myInterface closeRequest.
```

Add a 'Quit' button

Usually we create a specific button to close an interface. So, edit the `simpleSpec` of `MyInterface` and add a button 'Quit': with *'Quit'* in the field String and *'cerrar'* in the field Action:



Let's close the interface by specifying the *cerrar* method that will send a `closeRequest` to the GUI.

Performing Final Actions

It can be useful to control this closing and to execute specific treatments. Instead of the `closeRequest` action, define your own action, for example for `cerrar`:

```
cerrar
	self closeRequest.
	self doSomething.
```

As your interface inherits from **`AbstractGUI`**, the `close` method is already defined (it executes a `closeRequest`). So, instead of `cerrar`, you can just enter `#close` in the Action field of the 'Quit' button.

When a window has been asked to close, it first sends a `changeRequest` message to its model (ie. the interface). If it answers `false`, the window won't close; if it answers `true`, the window proceeds to close itself. Thus, the interface model has a chance to verify that no damage will be done if the window is closed.
For example, you can redefine the `changeRequest` method to confirm the user's intent to close the interface (but make sure that your method returns a boolean):

```
changeRequest
^super changeRequest
	ifFalse: [false]
	ifTrue: [Dialog confirm: 'Do you really wish to close?']
```

This notifies the window's model, so it can take precautions such as confirming the action.

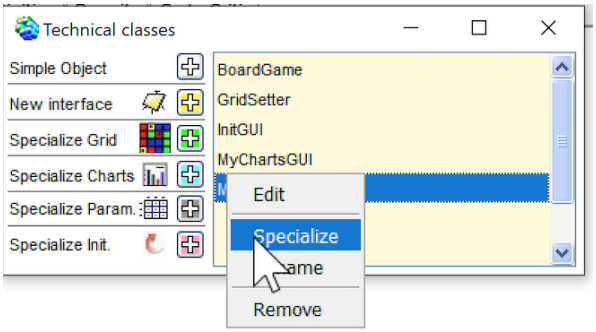
The configuration of a simulation from a GUI will be explained in the next session.

3- GUI for initializing an ABM

The organization of a simulation and the implementation of the agents are usually orchestrated during the initialization of the ABM.
For participatory modeling, it is often necessary to display a first interface when initializing a simulation, in order to configure the gaming session. The result of this interaction is mainly used to specify the number of players, their name and other characteristics.

Adding more elaborate widgets

The Adding basic widgets section presented simple widgets. Now, two more complicated widgets are described.
But first, let's specialize the MyInterface class to receive this dataset while reusing elements of MyInterface:



Call this new interface ‘`AgentsSetter`’.

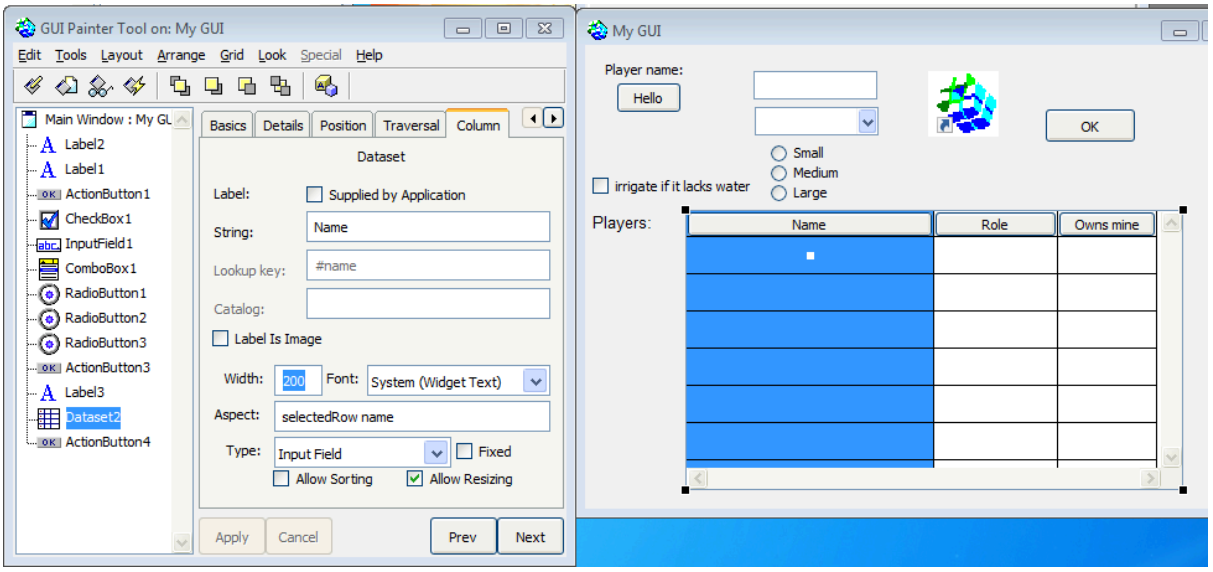
DataSet

A dataSet is a more complicated widget to set up. As a table, a dataset presents a list of similar objects (ex: Avatar) to edit. Datasets are similar to tables, but cells in a dataset can be edited directly by the user.
A dataset uses a `SelectionInList` attribute to hold the list of objects to be displayed, along with information about the current selection. Each object in the list is displayed in its own row, with individual attributes of the object displayed in their own columns. The property pages specify the means by which each column presents its data, whether using cells that contain read-only fields, editable fields, combo boxes, or checkboxes.

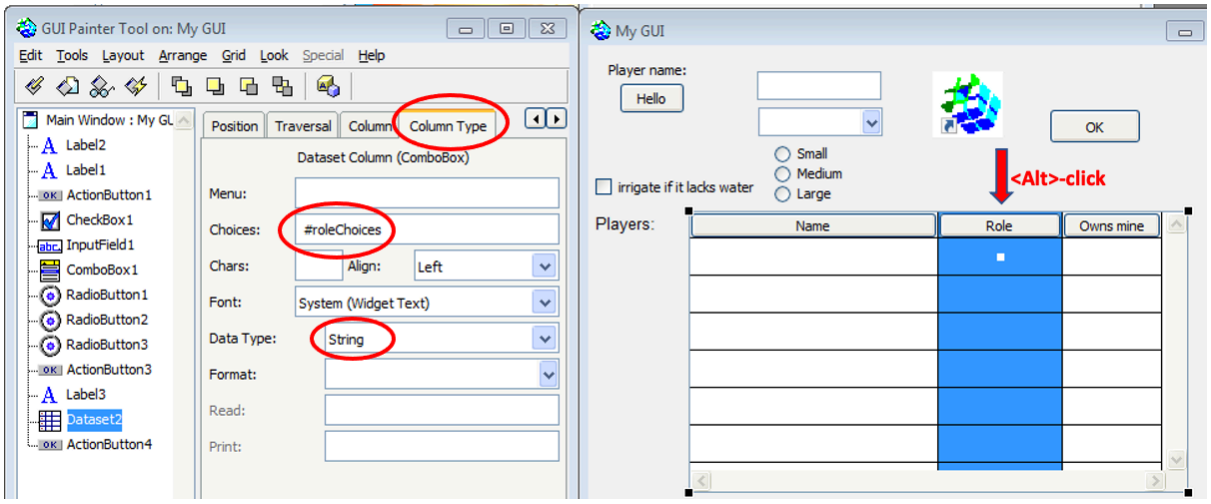
Main settings

Our example displays instances of Avatar, which has three attributes (**`name`**, **`role`**, and **`ownsMine`**), in three dataset columns. So, before, it is necessary to define these attributes for Avatar (by using the standard tool of Cormas). Then,

1. Add the **dataset** widget  to your canvas.
2. On the Basics property page, in the Aspect field, enter `playerList`.
3. Apply the property and install the interface (see Save your GUI) by saving this more elaborated interface as `#windowSpec` again.
4. Use ‘define’  to add the `playerList` attribute and its accessor method to your interface. The new `playerList` method returns a `SelectionInList` that will hold the list to be displayed.
5. In the dataset’s properties, click the **New Column** button for each column you want in the dataset. In the example, three columns are added.
6. On the dataset, **<Alt>-click** in the leftmost column to select it.
7. Click on the dataset’s *‘Column’* tab to set up the properties of the selected column. On the Column page,
    a. enter **Name** in the String field. This creates the title name of the selected column (which serves to display the avatars names).
    b. enter **selectedRow name** in the Aspect field. *selectedRow* refers to the value holder that will hold the `name`, `role`, and `ownsMine` of each Avatar. *selectedRow* will be another attribute of the interface.
    c. select **Input Field** as the Type. This causes each cell in the selected column to display its data in an editable input field.
    d. In the Width field, enter *200* to set the default width of this column (it can be resized by the user)

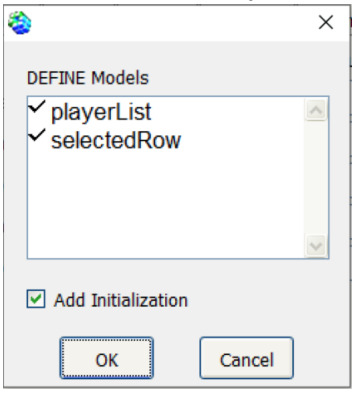


8. **<Alt>-click** on the middle column to select it. On the Column page,
   a. enter **role** in the String field and **selectedRow role** in the Aspect field. Select **Combo Box** as the Type.
   b. On the 'Column Type' page (next tab), enter **`roleChoices`** in the Choices field. So the ComboBox (we have already defined it with 4 possibilities) will be displayed for each row.
   c. As **`roleChoices`** returns a list of strings, set the Data Type field to *String*.



9. **<Alt>-click** on the rightmost column to select it. On the Column page,
   a. enter **Owns mine** in the String field (to set the column title) and
   b. **selectedRow ownsMine** as the Aspect.
   c. Select **Checkbox** as the Type.
10. When all properties have been applied, install the interface  .

11. Use the define  to automatically add the *selectedRow* attribute to your interface:



Keep the 2 selected attributes (`playerList` and `selectedTow`) because the accessor of `playerList` must be modified.
This will create the `selectedRow` accessor that returns a value holder for the line of the list that has been selected by the user.

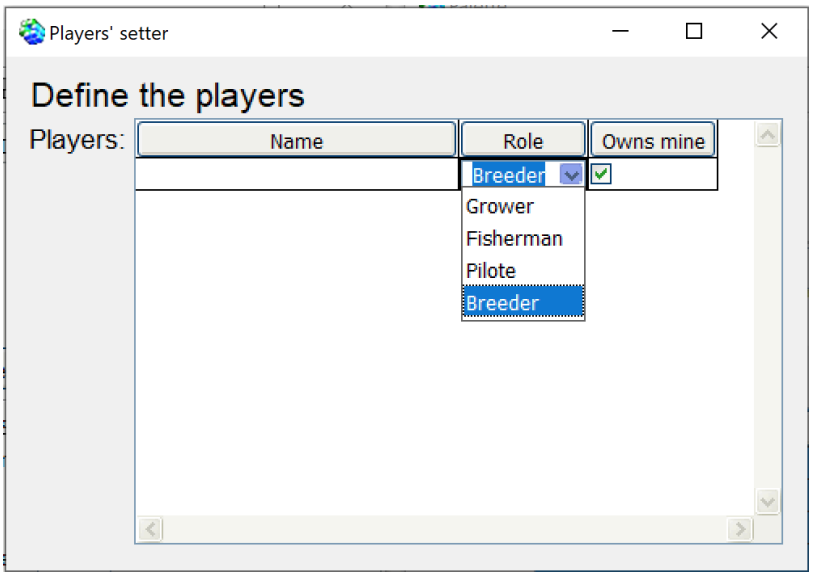
12. Use a browser to initialize the dataset:

```
initialize
 | listAvatars |

listAvatars := List new.
 listAvatars add: (Avatar basicNew name: '' ; role: 'Grower' ; ownsMine:
true ; yourself).
 self playerList list: listAvatars.
```

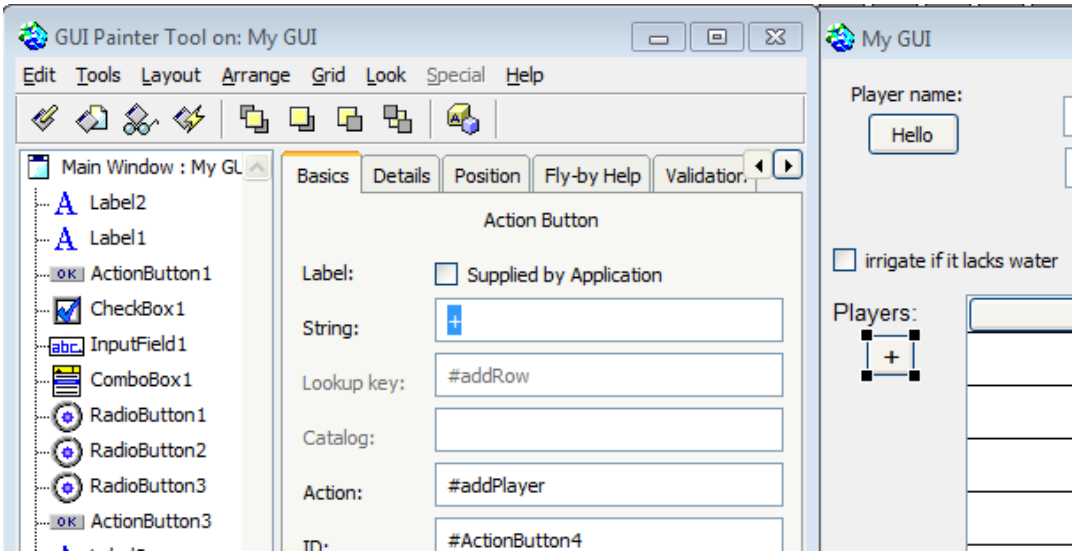
Note that, because we don't want to actually create new agents yet for the ABM (and not increment the ID of this class), `basicNew` is used instead of `new`. The concrete instantiation of the avatars will be done later, when closing the interface.

When you open the interface, the dataset contains one row. You can type a `name`, select a `role`, and change the `Owns mine` checkbox:



Add new players

In order to add new players (and new rows), it is necessary to create a specific button “+”. Add a button widget on your interface, then set its properties:



In the field Action, enter `addPlayer`. Install and Define this button.

Use the browser to set up the `addPlayer` method:

```
addPlayer
  "Add a new row on the dataset"
 self playerList list add: (Avatar basicNew).
```

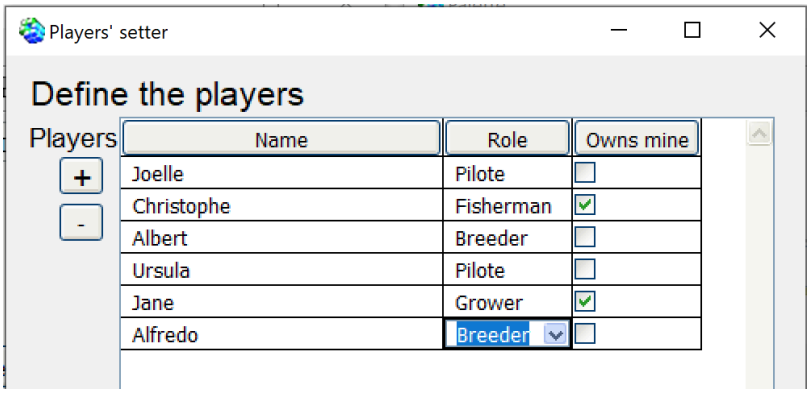
Remove players

In order to remove a player (and delete a row), we can create a specific button “-”. Add a button widget on your interface, then set its properties.

In the field Action, enter `removePlayer`. Install and Define this button.

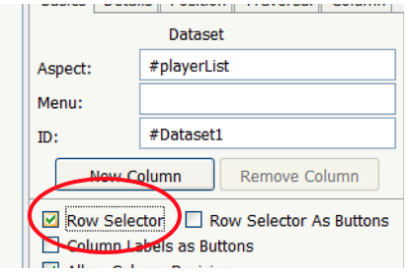
Use the browser to set up the `removePlayer` method:

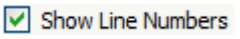
```
removePlayer
  "Delete a selected row of the dataset"
      | aRow |
      aRow := self selectedRow value.
      aRow isNil ifTrue: [^Dialog warn: 'Please, select a row.'].
      self playerList list remove: aRow.
```



Show line numbers

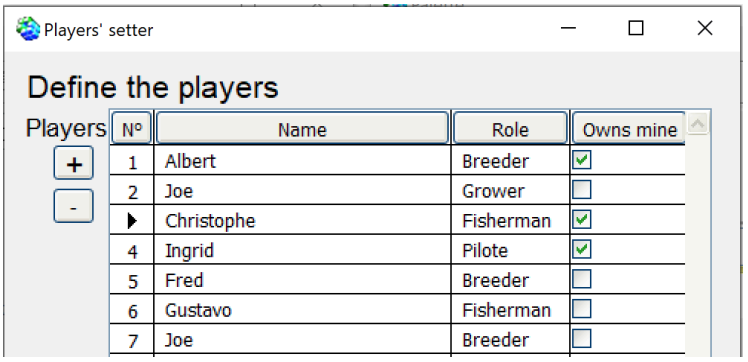
If you want a first column that shows the line number, click on *'Row Selector'* on the `'Basics'` tab:



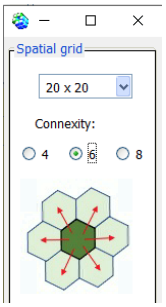
then 'Apply'. This will add a new column on the left side of the dataset.
Then on the `'Details'` tab, click on 'Show Line Numbers': 

.

On the dataset, **<Alt>-click** in the leftmost column to select it. Click on the dataset's *'Column'* tab and enter N° in the String field. This creates the title name of the first column. Apply and Install.



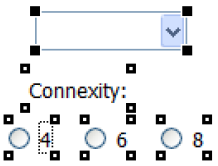
The section instantiating the agents from AgentsSetter explains how to open and use this interface.



ViewHolder

A ViewHolder allows you to display an image on your GUI. If the image doesn't change, it is much simpler to use Add image. But if the GUI must react to the user interactions by changing the image, you can use a ViewHolder.
Let's create a new interface to specify the grid of your model and give the new interface the name **`'GridSetter'`**.

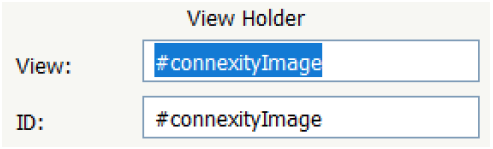
First, add a ComboBox (with `gridSize` for Aspect and `sizeChoices` for Choices fields) and 3 radio buttons for the attribute `connexity` (with Select field set to `#four`, `#six` and `#eight` for each one):



Main settings

To add a ViewHolder:

1. From the *Palette*, add a ViewHolder to your canvas (2nd line, 3rd button: ).
2. On the basics property page (ie. *GUI Tool*),
   a. in the field's View:, enter the name '`connexityImage`'. Copy it
   b. in the field's ID:, enter the same name (past).



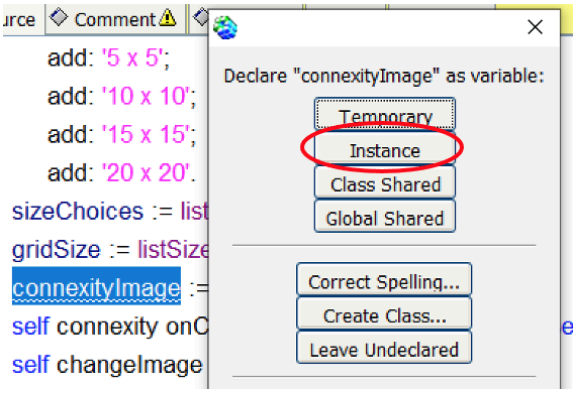
Then save.

Pay attention that the attribute `connexityImage` is not automatically done. We will create it when saving the initialisation, with this instruction:

```
connexityImage := MaskPainterView new.
```

```
initialize
	| listSize |
	super initialize.
	listSize := List new.
	listSize
		add: '5 x 5';
		add: '10 x 10';
		add: '15 x 15';
		add: '20 x 20'.
	sizeChoices := listSize asValue.
	gridSize := listSize first asValue.
	connexityImage := MaskPainterView new.
	self connexity onChangeSend: #changeImage to: self.
	self changeImage
```

When saving this code, specify that `connexityImage` must be an attribute:



Listening to changes

The `initialize` method has defined a hook on the attribute `connexity` through this instruction:

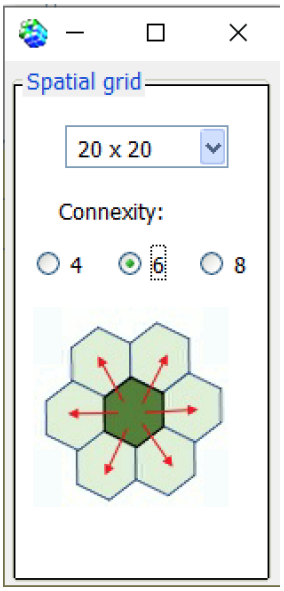
```
self connexity onChangeSend: #changeImage to: self
```

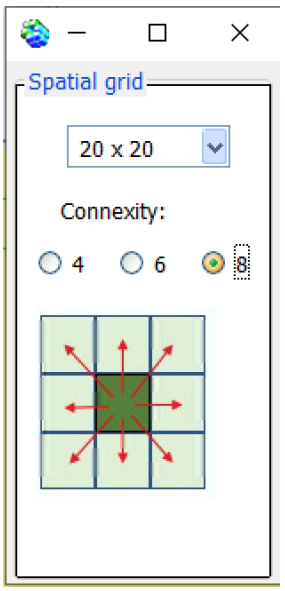
This enables the GUI to listen to any change on the attribute `connexity` and to react by executing the method `changeImage`. This method is as following:

```
changeImage
    | image |
    self connexity value = #four ifTrue: [image := self class fourNeighImage].
    self connexity value = #six ifTrue: [image := self class sixNeighImage].
    self connexity value = #eight ifTrue: [image := self class eightNeighImage].
    connexityImage model: image.
```

Note that in this case, 3 images have been installed at class level: *fourNeighImage*, *sixNeighImage* and *eightNeighImage*. See Load or capture an image by using `Cormas installImageUserRequest.`

Now, when opening the GridSetter interface, you should get a dynamic GUI that changes the image of the ViewHolder when the attribute *`connexity`* is changed by clicks on the radio buttons:

connexity 6: 

connexity 8: 

Opening an interface when initializing your model

During initialization, it is often necessary to open a first interface in order to configure the ABM. In this section, we will use the two previous GUI: `AgentsSetter` to instantiate the agents, then the `GridSetter` to specify the space model.

instantiating the agents from AgentsSetter

Here, we will reuse the **DataSet** widget explained in this section. The result of this interaction with **AgentsSetter** is used to specify the number of players, their name and other characteristics. This can be done in the `initAgentsFromAgentsSetter` of your model:

```
initAgentsFromAgentsSetter
    | myGUI listOfPlayers |
    myGUI := AgentsSetter new.
    myGUI model: self.
```

```
	myGUI open.
	self pauseSimulation. "Suspend the current simulation thread. To resume the
simulation, the #continueSimulation must be called when closing the interface"

	listOfPlayers := myGUI playerList listHolder value.
	listOfPlayers do:
		[:ag |
		(self newEntity: Avatar)
			name: ag name;
			role: ag role;
			ownsMine: ag ownsMine]
```

If the instruction `self pauseSimulation` is missing, then the simulation thread will execute all the initAgents method until the end of the initialization procedure without instantiating the agents.

Note that, because it inherits from `AbstractGUI`, **`AgentsSetter`** has an attribute called **'model'** that refers to your Cormas model. It will be used to advise the model to continue the simulation when the interface will be closed by the user (If you use an older version of Cormas, see Adding the attribute 'model' to the GUI in the Appendix).

Set the 'close' button of AgentsSetter

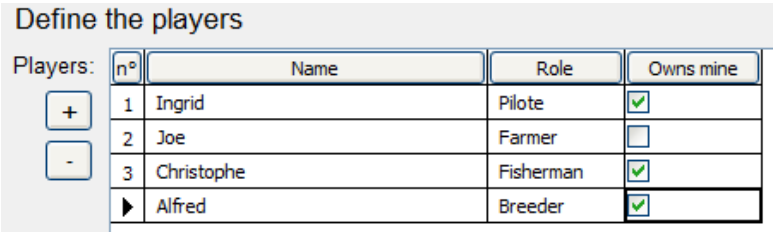
Add a 'OK' button on AgentsSetter GUI in order to close and continue the initialization phase of the simulation. Enter `closeAndGo` in the Action field, then accept and save.
Edit the `closeAndGo` method in a browser on `AbstractGUI`:

```
closeAndGo
	"Close the GUI and asks the model (cormasModel or entity) to resume the
current simulation"
	self closeRequest.
	self model isNil ifFalse: [model continueSimulation]. "Resume the suspended
simulation"
```

After closing the window, `continueSimulation` will resume the simulation that was suspended in the `initAgents` of the model.

Testing the model when closing the GUI

Now, when initializing the model (with `initAgentsFromAgentsSetterGUI`), the procedure will stop after opening the interface:



Then, after closing the window with the 'OK' button, the 4 agents will be effectively instantiated (`self newEntity: Avatar`) with the data given in the interface.

The Assigning a name and a color to the players section explains another way to set up the characteristics of the players by using sub-canvas instead of DataSets.

instantiating the Grid from GridSetter

Here, we will use the **'GridSetter'** interface to specify the space model: size of the grid and shape of the cells. But first, as GridSetter doesn't have a 'Quit' button, you must add it on its interface: set `#cerrar` in the Action field.

In your model, add the `initSpaceFromGridSetter` method:

```
initSpaceFromGridSetter
	| gridGUI |
	gridGUI := GridSetter new.
	gridGUI model: self.
	gridGUI open.
	self pauseSimulation. "Suspend the current simulation thread. To resume the
simulation, the #continueSimulation must be called when closing the interface"

	self	createGridX: gridGUI numLines
		Y: gridGUI numColumns
		neighbourhood: gridGUI connexityAsNumber
		closed: true.
```

In the next section, we will reuse the previous interfaces in order to create a full interface that combines `GridSetter` and `AgentsSetter`.

SubCanvas: reuse the previous canvas

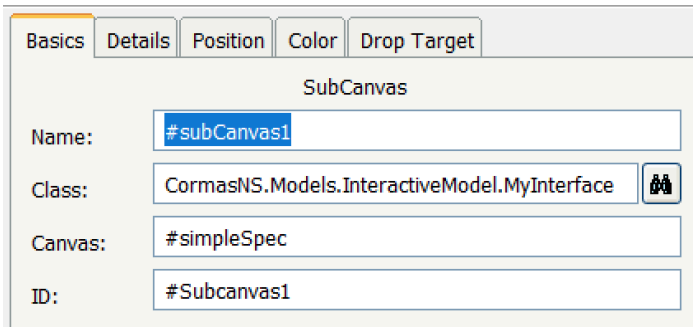
We can reuse the previous interfaces by arranging them in a new interface. Here we will create a GUI to initialize the model with grid and agent parameters specified by the user.

Creating a new GUI with 3 subcanvas

Create a new GUI called **`InitGUI`**. Edit the `windowSpec`.

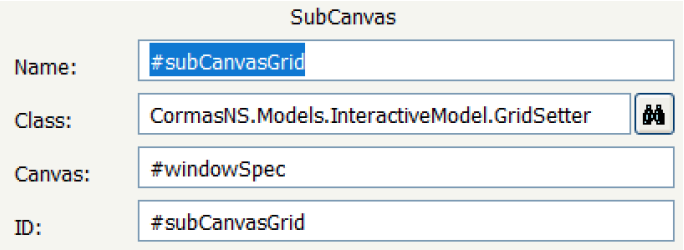
First, add a simple 'OK' Button (with *`closeAndGo`* for Action field).
Then add 2 subcanvas as for the **2 players**:

1. From the *Palette*, add a SubCanvas to your interface (2nd line, 4th button: ).
2. On the basics property page (ie. *GUI Tool*),
   a. in the field's Name:, enter the name *`subCanvas1`*. (Copy it)
   b. in the field's ID:, enter the same name (past)
   c. in the field's Class:, enter the class that stores the canvas. In our case, *`MyInterface`*
   d. in the field's Canvas:, enter the method name that specifies the canvas. In our case, `simpleSpec`

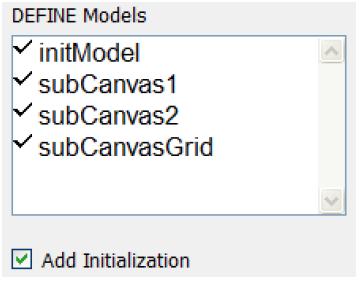


Repeat this procedure for the second subcanvas with the same fields except for Name, ID: enter *`subCanvas2`*.
Add a third subcanvas for the **grid setter**: repeat the previous procedure:

3. On the basics property page (ie. *GUI Tool*),
    a. in the field's Name:, enter the name *`subCanvasGrid`*. (Copy it)
    b. in the field's ID:, enter the same name (past)
    c. in the field's Class:, enter the class that stores the canvas. In our case, *`GridSetter`*
    d. in the field's Canvas:, enter the method name that specifies the canvas. In our case, `windowSpec`



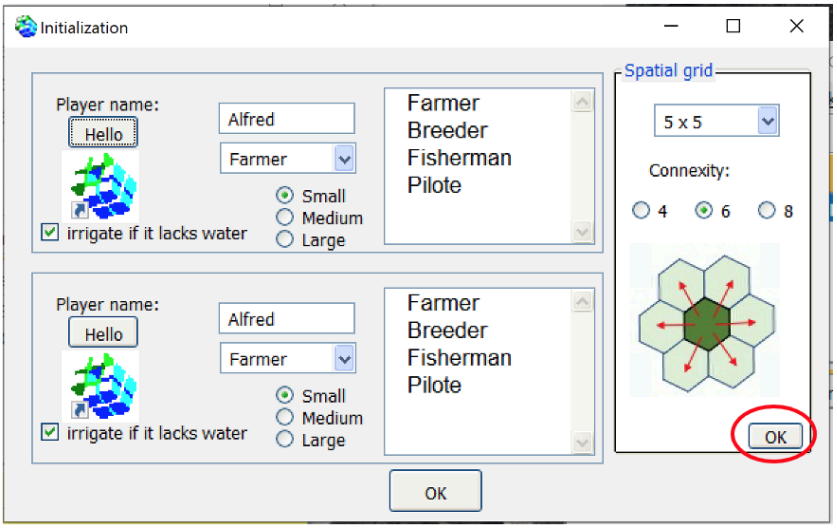
Then save and 'Define' the attributes (button  of the GUI Tool):



Adapting the canvas

Hide a widget

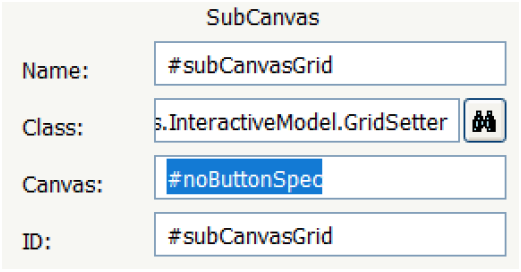
A click on the 'Open' button should instantiate this GUI, like:



Here, we have 2 'OK' buttons. The one on the grid specifications should be removed. There are several options for this: either,

1. remove this button from the `GridSetter` specification, then simply save it (but in this case the **`initSpaceFromGUI`** won't work anymore), or

2. remove this button and rename the specification to *noButtonSpec* (and change *subCanvasGrid* with the Canvas field set to `noButtonSpec`)



3. keep the interface as it is, but without displaying the button when the interface is opened.

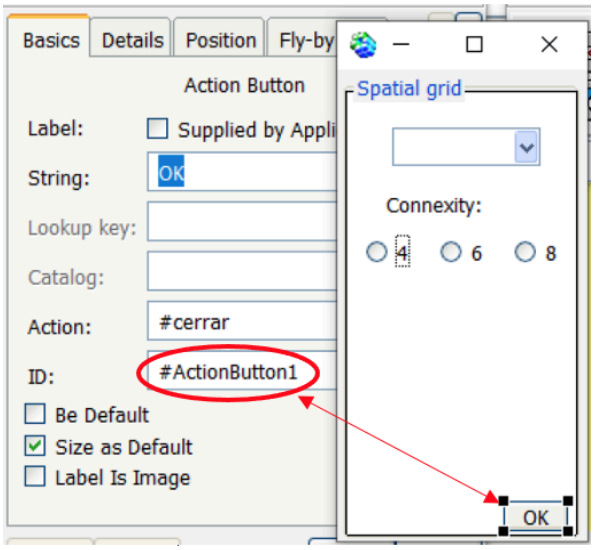
Here we propose the third option that is the most complicated, but it helps to understand how to modify the elements of an interface when it is already open. We access them from their IDs.

`init2agentsWithSubcanvasGUI`

```
| initGUI bldr |
initGUI := InitGUI new.
initGUI model: self.
bldr := initGUI allButOpenInterface: #windowSpec.
(initGUI subCanvasGrid builder componentAt: #ActionButton1)
	disable;
	beInvisible.
bldr open.
self pauseSimulation.

self
	createGridX: initGUI subCanvasGrid numLines
	Y: initGUI subCanvasGrid numColumns
	neighbourhood: initGUI subCanvasGrid connexityAsNumber
	closed: true.
(self initAgentFrom: initGUI subCanvas1) moveTo: self topRightCell.
(self initAgentFrom: initGUI subCanvas2) moveTo: self bottomLeftCell.
self createFarms
```

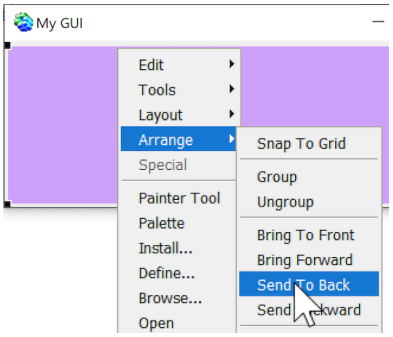
The instruction `(initGUI subCanvasGrid builder componentAt: #ActionButton1)` returns the visual component (`WidgetWrapper`) with the **ID** = `#ActionButton1`. This component is only accessible through the **builder** of the interface.



This visual component can execute several useful messages: `disable, beInvisible, enable, beVisible, moveTo: aPoint, …`
Note that the `subCanvasGrid` builder is a distinct instance than the one of `initGUI` builder.

Set distinct colors for the players

If we want different colors for the player subcanvas, we can change the background color. But, as the background of canvas doesn't display its color (an error of VW?), a *region* widget  must be added on the GUI and pushed behind all the other widgets. Set `#bkColor` as the **ID** of the region. To arrange the relative position of a widget, right click on the widget (in the UI Painter), then select *Arrange → Send To Back*



In order to dynamically change the color of this region, a method must be defined. Like for hiding the button (previously), `setBackgroundColor:` (of `OneAgentSetter`) requires access to the **ID** of the widget. To do this, it uses the **Builder** which is automatically provided to the interface at the display stage:

```
setBackgroundColor: aColorValue

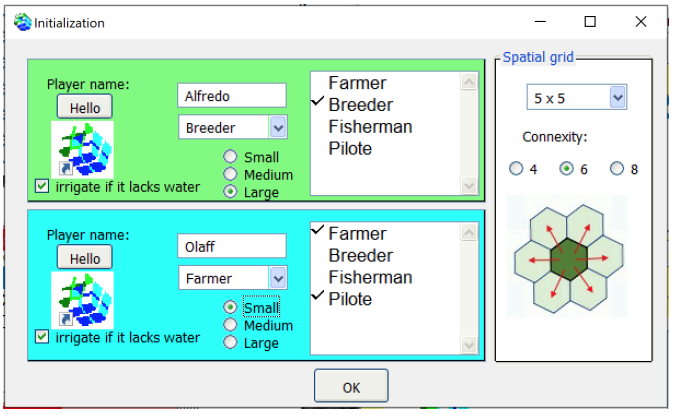
        | aWrapper lookPref |
        aColorValue isNil ifTrue: [^self].
        aWrapper := self builder componentAt: #bkColor.
        lookPref := aWrapper lookPreferences backgroundColor: aColorValue.
        aWrapper lookPreferences: lookPref.
```

We must now redefine the method `postBuildWith:` for the `InitGUI` class:

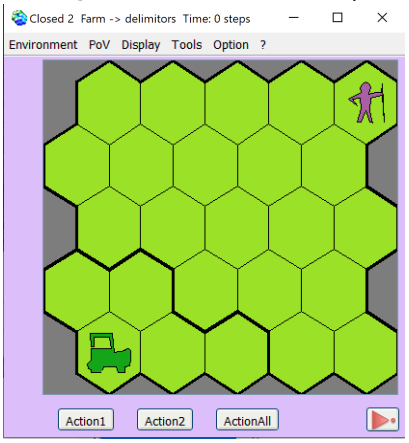
```
postBuildWith: aBuilder
        "This message is sent by the builder when it has completed work on
        either a complete window or a SubCanvas."
```

```
self subCanvas1 setBackgroundColor: ColorValue paleGreen.
self subCanvas2 setBackgroundColor: ColorValue cyan.
```

The new interface should look like:



At the end of the initialization (`init2agentsWithSubcanvasGUI`), the grid should look like:



4- GUI for one agent

Open the GUI for one agent

It is also possible to open a GUI at any time during the simulation and for example to propose a decision at each step. It is also possible to open a GUI on an agent, for example from the conceptual menu. This is what we will do here, with `MyInterface`.
As we want this interface to open from the spatial grid by clicking on an agent, we can redefine the method `methodsForControl` at the class level of `Avatar`:

```
methodsForControl
	^#(#openGUI #step)
```

We can also redefine (at instance level) `reactToDoubleClick` in this way:

```
reactToDoubleClick
"Purpose: By default when this entity receives a double click event, it opens an inspector.
This method can be redefined at your model level to perform a specific method. "
	self openGUI
```

And we must define the `openGUI` method:

```
openGUI
	| initGUI |
	initGUI := MyInterface new.
	initGUI initializeFor: self.
	initGUI openInterface: #windowSpecB.
```

Then the `initializeFor:` that will set the agent as the `model` of this interface:

```
initializeFor: anAgent
	"Set the interface for anAgent."
	self model: anAgent.
	self playerName value: anAgent name.
	self role value: anAgent roleName.
	self doIrrigate value: anAgent doIrrigate.
	self familySize value: anAgent familySize.
	self goodSelections selections:
		(anAgent goods collect: [:g | g class name asString]).
```

Note that the `initialize` method is automatically sent when `MyInterface` is instantiated. So, the GUI will open with its fields set to the targeted agent.

Close GUI and change the agent's attributes

The 'Quit' button executes the method `close` (defined in `AbstractGUI`). It just closes the GUI. In order to change the attributes of the agent, we must redefine this method for `MyInterface`:

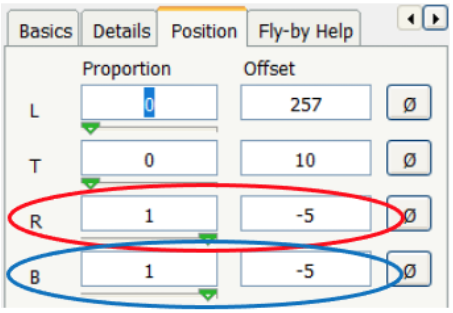
```
close
	super close.
	(self model isKindOf: Avatar) ifFalse:[^nil].
	"model = an Avatar"
	model name: self playerName value.
	model setRoleFromString: self role value.
	model familySize: self familySize value.
	model doIrrigate: self doIrrigate value.
	self goodSelections selectionDo: [:st | model buyGood: st].
```

The instruction '`super close`' will execute the method `close` defined at the super class level (ie. `AbstractGUI`), that closes effectively the GUI. Then the modifications set by the user on the GUI are used to modify the targeted agent (ie. the model of the GUI).

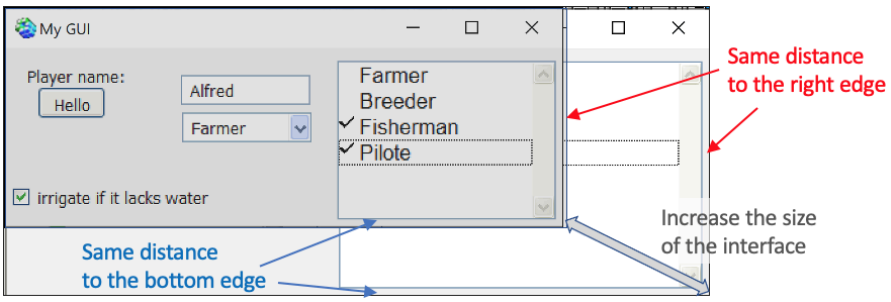
5- Advanced settings

Adapting the widgets when the GUI size is changed

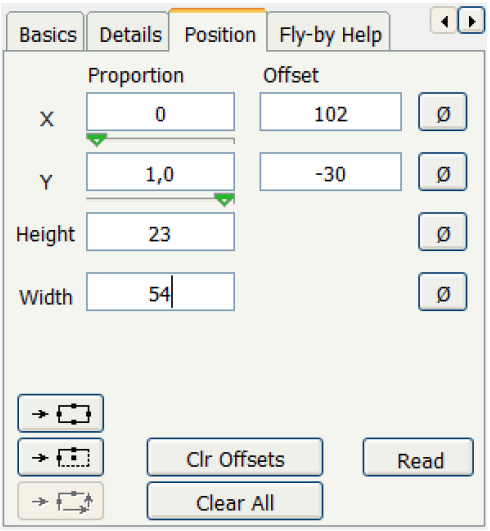
If you want a widget (for example, the list of `MyInterface`) to adapt when the user expands the interface, you have to change the characteristics of this widget. Select the '`Position`' item of the *GUI Painter Tool*. It presents the positions of the 4 edges of the widget: **L** (left), **T** (top), **R** (right) and **B** (bottom). Set **R** and **B** proportions to **1** (100%) and their Offset to **-5** (pixels), as the following:



This means that the bottom-left point of the list will remain at 5 pixels *before* the bottom and right edges of the window:



The same principle applies to the other widgets. For example, if you want the 'Quit' button to remain at the bottom of the window even when it is enlarged, you have to change the Position in this way:



The Y proportion is set to 1, with an offset of -30, and the height and width of the button remain unchanged (23 and 54 pixels).

Listening to events

onChangeSend:to:

When an attribute (mainly a ValueHolder) of the GUI has been changed by the user, it can be useful to execute a method to react to this interaction (to check for instance if the new value specified by the user is correct). For that, add an instruction at the end of the `initialization` in order to listen to the change events on this attribute, thus to trigger a specific method. The GridSetter GUI uses this procedure: when the user select a new connexity, then `changeImage` is triggered:

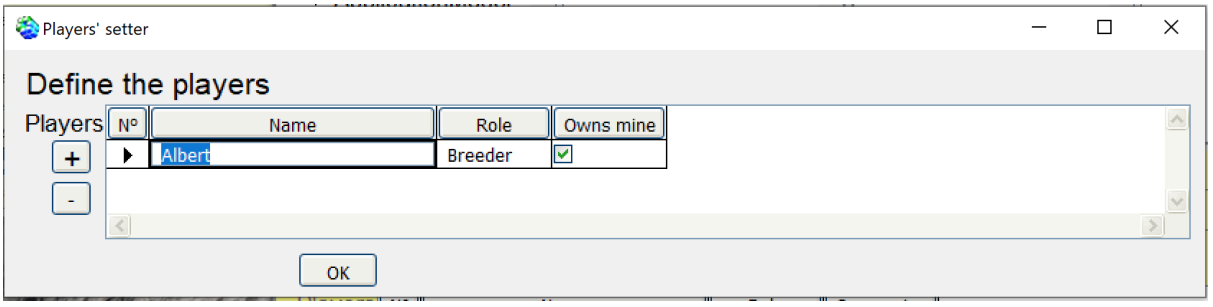
```
initialize
```

```
        super initialize.
        self connexity onChangeSend: #changeImage to: self.
```

This enables the GUI to listen to any change on the attribute `connexity` and to react by executing the method `changeImage`.

Adapting the dataset

Even if the `playerList` of the `AgentsStetter` (see DataSet) is correctly adapted to change its shape when the window is changed, its content (the columns) are not automatically updated:



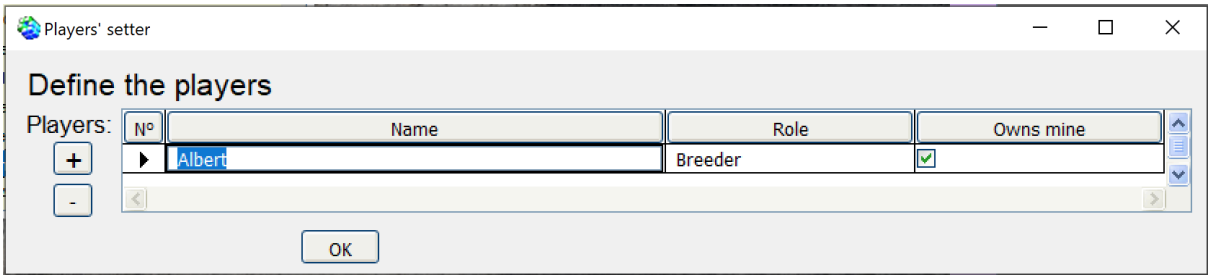
To modify the size of the columns, `AgentsStetter` must listen to the `resize` events.

```
postBuildWith: aBuilder
        aBuilder window whenAny: #(#resize) send: #windowHasResized to: self.
        self windowHasResized.
```

Then let's define the `windowHasResized` method:

```
windowHasResized
        "Set the columns' width when size of the window has been resized"
        | aDatasetobject columnWidth |
        ((self wrapperAt: #Dataset1) isNil not) ifFalse: [^nil].
        aDatasetobject := self widgetAt: #Dataset1.
        columnWidth := (builder window width - 80 "Offset: L: 70, R: 10" - 30 "col
N°" -30 "Scroller") / 4.
        (aDatasetobject columnDescriptors at: 1) width: 30 "N°".
        (aDatasetobject columnDescriptors at: 2) width: (columnWidth * 2) "Name".
        (aDatasetobject columnDescriptors at: 3) width: columnWidth "Role".
        (aDatasetobject columnDescriptors at: 4) width: columnWidth "Owns mine".
        "refresh display"
        aDatasetobject computeStarts.
        aDatasetobject invalidateRectangle: aDatasetobject bounds.
```

As the size of the first column remains constant (30 pixels), the size calculation is done for the other three columns. Furthermore, as we want the size of the "Name" column to be twice as large as the others, the total size of the widget is divided by 4 to obtain the `columnWidth`.



Automatically change the cashbox value

Let's specialize `MyInterface` as `OneAgentDecision`.
So create the new class, then drag the `#windowSpecB` method from `MyInterface` to `OneAgentDecision` in order to redefine it. Add a label '`cashbox`' and an input field 'cashbox' with type = `Number`.



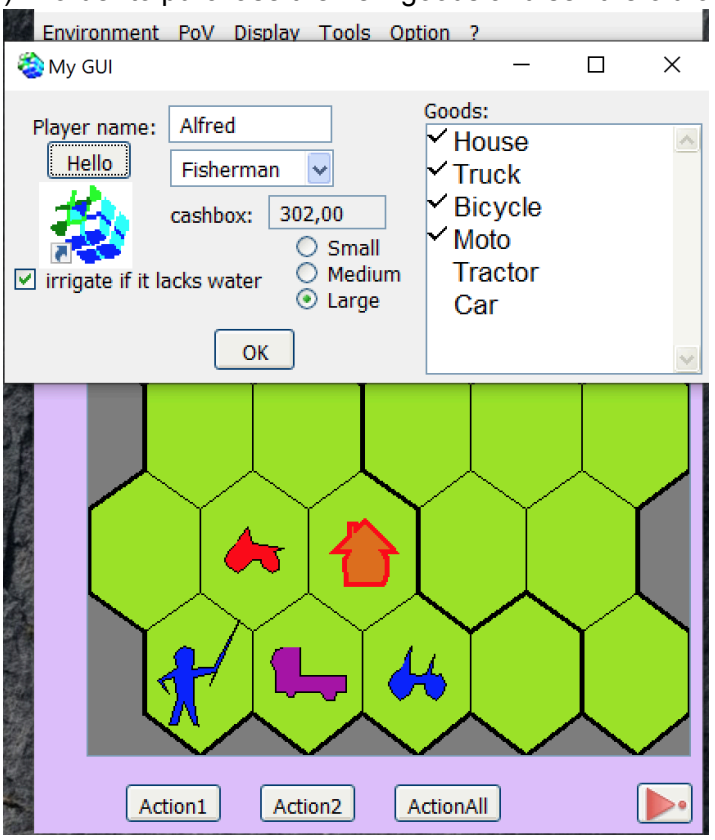
Here we want the cashbox to listen to the events in the list `goodSelections` and automatically adjust its value according to the selected goods.

With:

`changeCashboxForAgent`

```
self model adaptGoodsTo: self goodSelections selections.
self cashbox value: self model cashbox
```

Each time a selection in goodSelections is done, the `changeCashboxForAgent` method is triggered and the method `adaptGoodsTo:` is executed by the targeted agent (see this method in the code) in order to purchase the new goods and sell the old ones.



Automatically change the background

At the end of the `initializeFor: anAgent` method, add the following instruction in order to listen to any change of role:

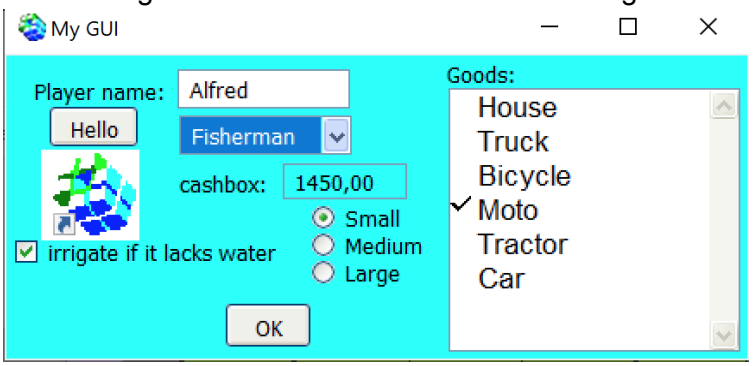
```
self role onChangeSend: #changeColor to: self.
```

Then define the changeColor method:

changeColor

```
	self setBackgroundColor: (self model class classNamed: self role value asSymbol) defaultColor
```

Thus, the interface will change its color each time the role is changed:



6- Examples of ABM interfaces

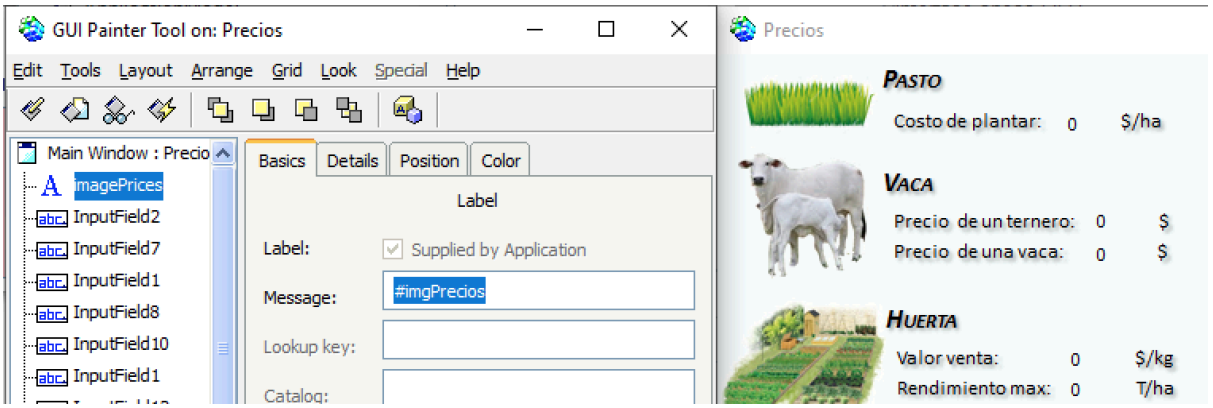
The `Universal Example` (see Predefined examples of widgets) is a complete interface example with several views on the same models. It could be useful to have a look. Most of the following examples are available in the "`InteractiveModel`" toy model.

Displaying parameters: prices and costs

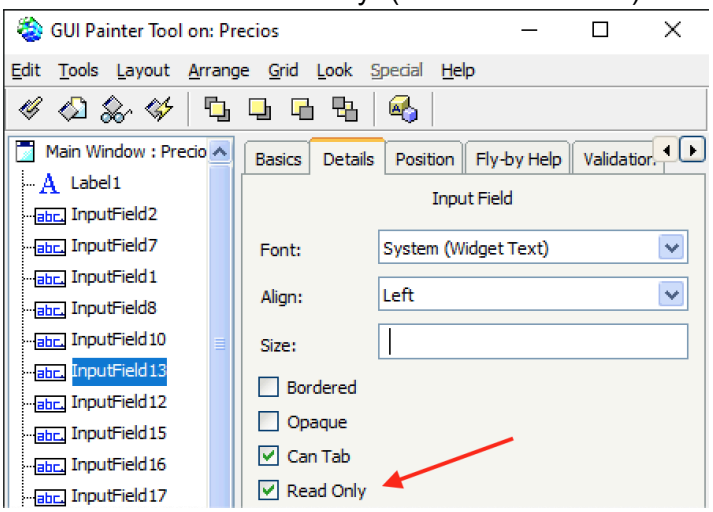
ContamiCuenca (code available on Cormas web site) offers a simple interface (called *'Precios'*) indicating several costs and prices of the entities of the model:



This interface is based on an image built outside Cormas (from Powerpoint for example). The interface is just composed by a background image, *#imgPrecios,* stored at the class level (to load an external image, see this chapter). A set of inputFields are positioned in front:



Note that these *inputFields* are set to “read only” (in the ‘Details’ tab):



To open this ‘Precios’ interface:

```
abrirPrecios

    | interfaceDecision |
    interfaceDecision := InterfaceDecision new.
    interfaceDecision initPrecios.
    ^interfaceDecision openInterface: #precios.
```

The `#initPrecios` method set the value of all the inputFields:

```
initPrecios
    self pastoCostoPlantar value: Pasto costoPlantar.
    self precioTernero value: ActorRural precioTernero.
    self precioVaca value: ActorRural precioVaca.
    self huertaPrecioVenta value: Huerta precio.
    .....
```

Each InputField (the View) of the interface is managed by an instance of ValueHolder (the Model: `pastoCostoPlantar, precioTernero, precioVaca, ...`) which can be changed by sending the `value:` message. In this example, each ValueHolder value is initialized with the value of a class variable (`Pasto costoPlantar, ActorRural precioTernero, ...`).

Instead of creating a new parameters interface, it is also possible to specialize the parameters table proposed by default in Cormas (see this section).

Assigning a name and a color to the players

MineSet (code available on Cormas web site) offers a interface (called *Initialization*) where it is possible to change the name of the players and to assign a color for each one:

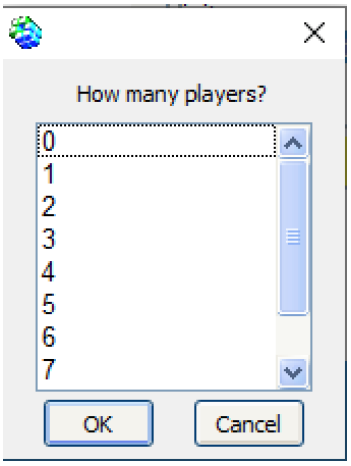


This interface is a bit more complicated (but cleaner) as it uses **sub-canvas** (instead of DataSet) that are added to the main interface.

`initAgents`

```
      | settingInterface nbPlayers |

"First: defining the number of players"
nbPlayers := Dialog
            choose: 'How many players?'
            fromList: #('0' '1' '2' '3' '4' '5' '6' '7' '8' '9')
            values: #(0 1 2 3 4 5 6 7 8 9)
            lines: 8
            cancel: [].
nbPlayers ifNil: [^nil].
“2nd: setting the players interface”
settingInterface := UIGame new.
settingInterface model: self.
settingInterface initializePlayers: nbPlayers.
settingInterface allButOpenInterface: ('initSpec' , nbPlayers asString) asSymbol.
settingInterface setColorOfPlayerCanvas.
settingInterface finallyOpen.
“3nd: pause the simulation”
self pauseSimulation.
```



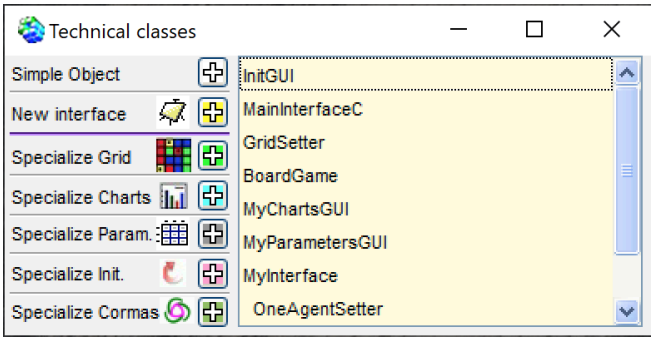
Here the **UIGame** interface is in charge of displaying the players' name and their color. UIGame has an attribute called 'model' that refers to your CormasModel. When closing the interface, it will be able to specify the players (name, type and color), then to continue the simulation. Indeed, there is a time management via the methods `pauseSimulation` (that stops the execution flow) and `continueSimulation` (that resumes the thread).

```
initializePlayers: numPlayers

	| defaultNames aPlayer |
	self initialize.
	nbPlayers := num.
	defaultNames := #('Yellow' 'Cyan' 'Green' 'Red' 'Magenta'  'Orange' 'Pink'
'Olive' 'Blue').
	1 to: numPlayers do:
			[:i |
			aPlayer := self perform: ('p' , i asString) asSymbol.
			aPlayer builder: nil.
			aPlayer playerNumber value: i.
			aPlayer companyName value: (defaultNames at: i)]
```

Specializing the built-in interfaces of Cormas

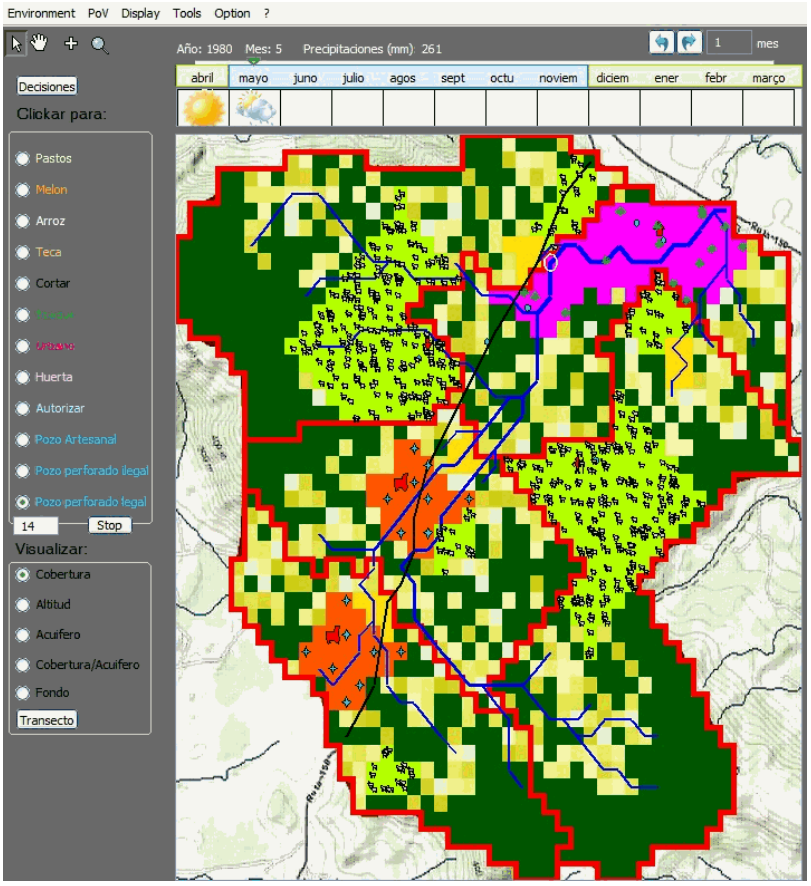
As explained in the section Using the Cormas Toolbox, Cormas (v. 2022.3) offers a set of tools for technical classes to facilitate the design of new interfaces and to integrate them in your model:



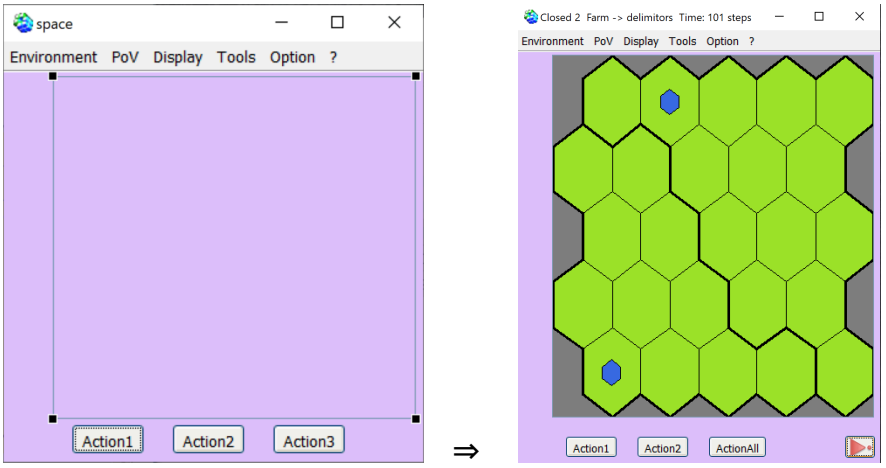
Then, by clicking on the various [+] buttons, you can create new classes such as SimpleObject or New interface. You can also specialize the built-in interfaces of Cormas, such as: the spatial grid, the charts of the probes, the parameters, the initialization interface or Cormas itself. This toolbox avoids the quite tedious configuration of these technical classes to integrate them in your model (Namespace and Package).

Spatial grid as Game board

Cormas offers a default spatial interface. It is possible to modify this grid to adapt it to your needs. It is usually open from the Cormas menu `Visualisation` → `Spatial grid`.
Example of game board in the *ContamiCuenca* model:



To specialize the **SpatialInterface** class, click on the green + button  . Then give a name to this new class (ex: *BoardGame*). Now, you can modify `BoardGame` as you want:



When creating the new class for the spatial grid, the method `spaceInterfaceType` is automatically redefined at class level of your CormasModel (e.g. *InteractiveModel*), as follows:

```
spaceInterfaceType
```

```
	"Return the class of the Space interface that should be instantiated
when opening the spatial grid.
	This method has been generated by Cormas"
		^ BoardGame
```

So, when opening a new grid, Cormas will create a new instance of `BoardGame` and display it.

Adapting the charts

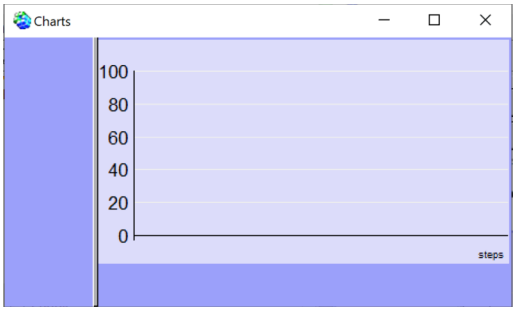
Cormas offers a default charts' interface. As previously, it is possible to modify this interface to adapt it to your needs. It is open via the Cormas menu `Visualisation` → `Charts`.

To specialize the **ChartsInterface** class, click on the blue [+] button  . Then give a name to this new class (ex: *MyChartsGUI*). Now, you can modify `MyChartsGUI` as you want.

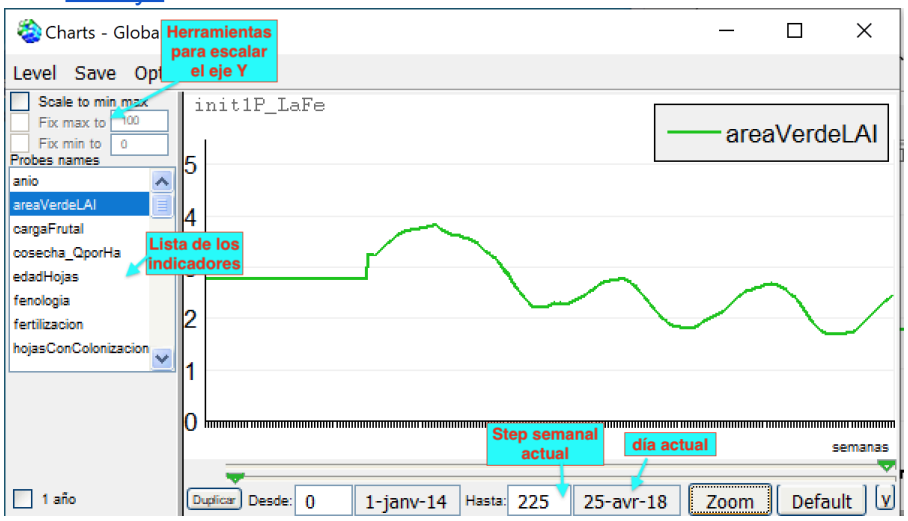
In the main class of your model, the method `ChartsInterfaceType` has been automatically redefined as follows:

```
	chartsInterfaceType
	"Return the class of the Charts interface that should be
instantiated when opening a chart view.
	This method has been generated by Cormas"
		^ MyChartsGUI
```

Example of charts' interface for InteractiveModel:



Example for the MiRoya model:



In this specialization of the **ChartsInterface**, a CheckBox ("1 año") has been added at the bottom-left. It shows the curves only for the last year. In addition to the standard steps (integers), the time is presented in the form of dates. Finally, a button ("y") has been added at the bottom right to show the curves truncated in one-year sections.

Adapting the parameters' interface

Cormas offers a default parameters interface. It opens by clicking on the 'Parameters' button  .

As previously, it is possible to modify this interface to adapt it to your needs.

To specialize the **ParameterInterface** class, click on the gray [+] button  . Then give a name to this new class (ex: *MyParametersGUI*). Now, you can modify `MyParametersGUI` as you want.
In the main class of your model, the method `ParameterInterfaceType` has been automatically redefined as follows:

```
parameterInterfaceType
        ^ MyParametersGUI
```

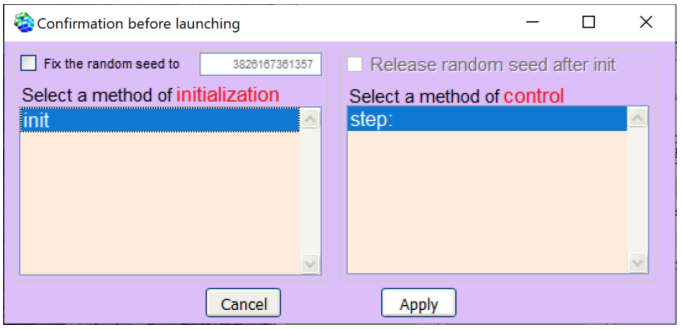
Adapting the Scenario builder interface (SimManager)

Cormas offers a default Scenario builder interface (coded into SimManager) to set up the initialization of a model. It opens by clicking on the 'Initialize' button of Cormas:



As previously, it is possible to modify this interface to adapt it to your needs.

To specialize the **SimManager** class, click on the pink [+] button  . Then give a name to this new class (ex: *MyScenarioBuilderInterface*). Now, you can modify `MyScenarioBuilderInterface` as you want. Ex:



In the main class of your model, the method `ParameterInterfaceType` has been automatically redefined as follows:

```
simManagerType
	"Return the class of the SimManager interface that should be instantiated when opening a scenario Builder Interface.
	This method has been generated by Cormas"
	^ MyScenarioBuilderInterface
```

Adapting Cormas' interface

Cormas comes with a default interface. As previously, it is possible to modify this interface to adapt it to your needs.

To specialize the **Cormas** class, click on the green [+] button  . Then give a name to this new class (ex: *MyCormas*). Now, you can modify `MyCormas` as you want.

In the main class of your model, the method `cormasType` has been automatically redefined as follows:

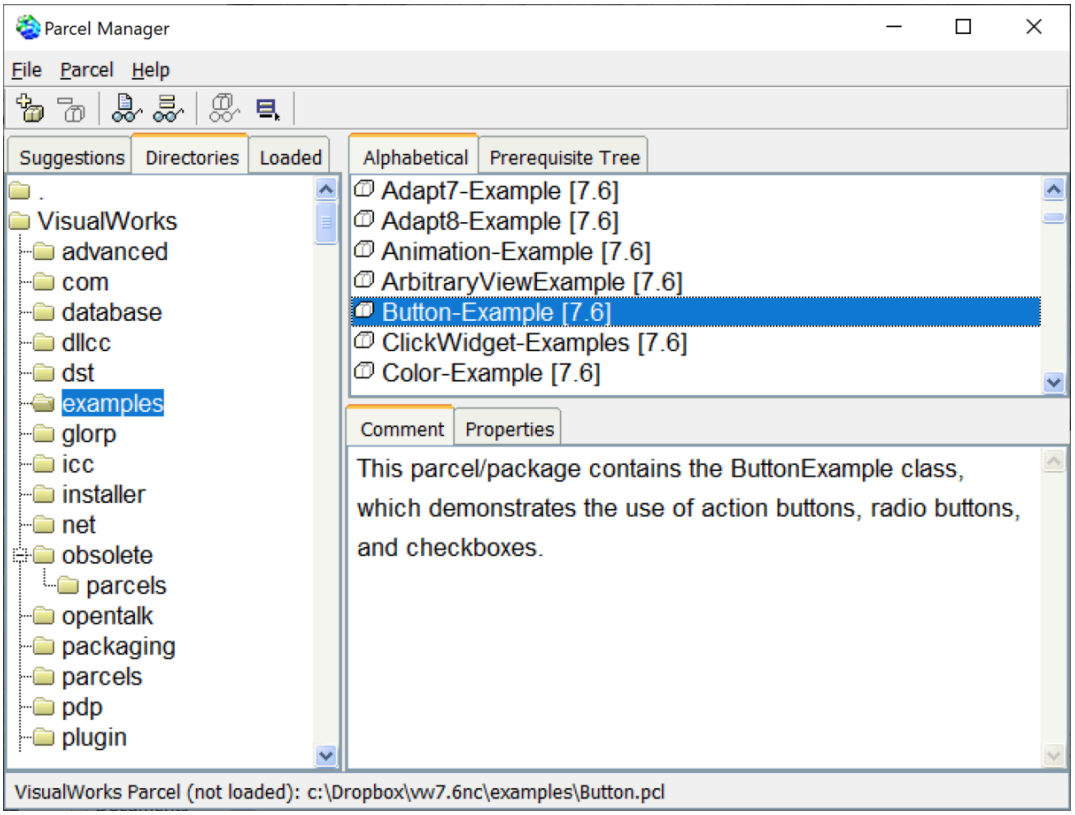
```
cormasType
        ^ MyCormas
```

The computational aspect of this option is more complicated: indeed, the already active Cormas instance must be replaced by a new instance of `MyCormas`. To see your new interface, you have to save and reload the model.

Appendix

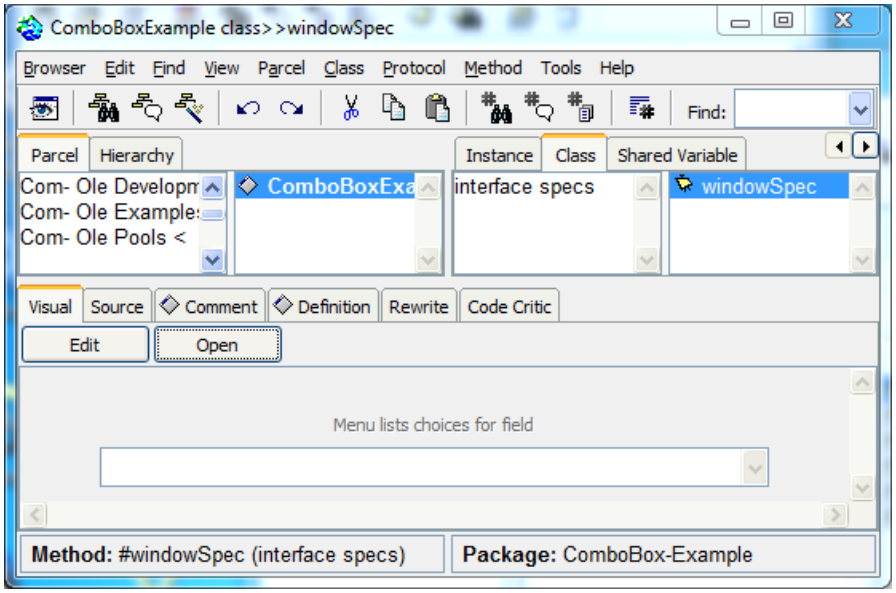
Loading VW examples of GUI

To load a VW example, open a **Parcel Manager** (2nd button of the VW main interface ), then select the 2nd item *'Directories'* and *'examples'*:

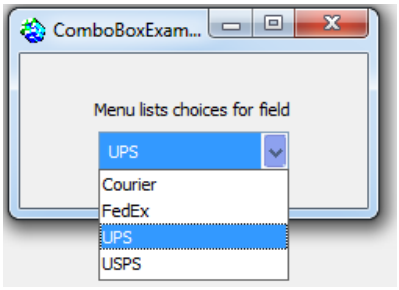


If the examples' folder is not available on your disk, download and unzip this file (examples.zip) into cormas/parcels/.

Once the examples' Parcel is loaded, click on the 'Browser' button  to edit it.
Select the *Example* class (ex: ComboBoxExample) and the 'Class' tab, then the **`windowSpec`** method:



- A click on the **'Open'** button will instantiate this interface:



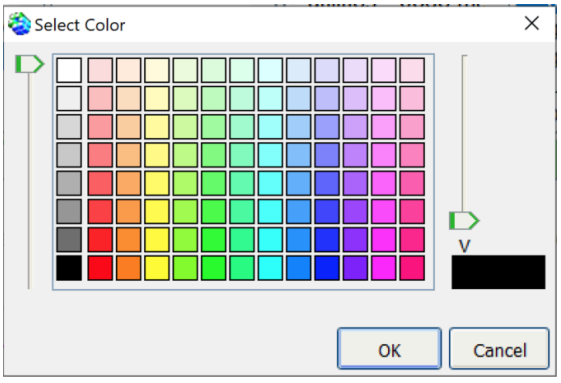
- A click on the **'Edit'** button will open the painter tools (see later).

Simple dialog vs ApplicationModel

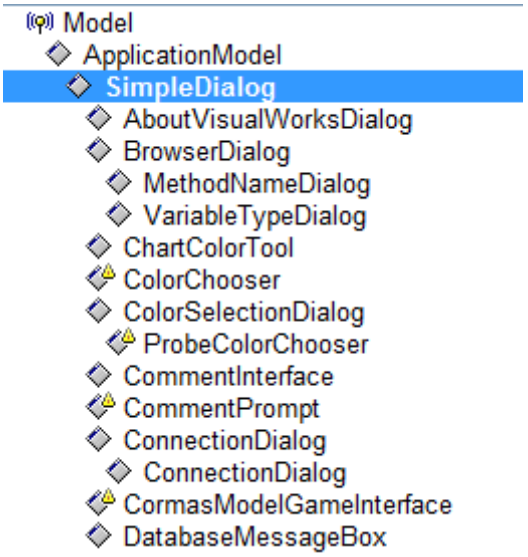
Each *Dialog* interface is pre-emptive: it holds the control until it is closed. In that case, the user is not authorized to click on another interface. The Dialog interface must provide a Cancel button, an OK button, or the equivalent.

using Dialog or sub-classes of SimpleDialog

By using **Dialog** or sub-classes of **SimpleDialog** (like in 1- Dialog: using predefined simple GUIs), there is no need to pause and resume the simulation (`pauseSimulation` and `continueSimulation`), because the execution flow is suspended until the dialog is closed. It is therefore simpler to implement, but at the same time it reduces the possibilities for the users: this Dialog interface prevents them from checking other information. This is for example the case of the ColorChooser (that can be called this way: `Cormas colorChooser`):



`colorChooser` uses ColorSelectionDialog that is a subclass of **SimpleDialog** like many other interfaces:



All SimpleDialog subclasses are pre-emptive interfaces that hold the control until they are closed.
Because they prevent the users from doing anything else, they must be chosen with care.
For the majority of your interfaces, we recommend to inherit your interface from **ApplicationModel** (that is not pre-emptive) as presented in Designing a GUI or Examples of ABM interfaces and to control the execution flow with `pauseSimulation` and `continueSimulation`.

Turning a graphical interface into a Dialogue

Nevertheless, if you want to transform a simple GUI as a Dialog interface, you can send the message `openDialogInterface: #guiName` (instead of `openInterface: #guiName`). In that case, the guiName interface will hold the control until it is closed.
In order to close this preemptive interface, the 'OK' button must have `#accept` as Action (in the action field) (and the 'Cancel' button `#cancel`), with:

```
accept
	self closeRequest.
	^true

cancel
	self closeRequest.
	^false
```

Load or capture an image

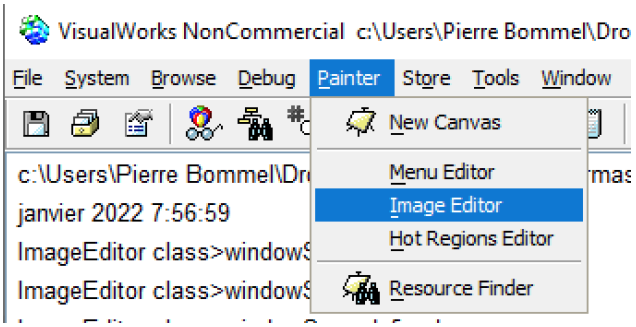
To load an external image (jpg or png) and to store it as a method, there are 2 possibilities:

1. by loading a JPG or PNG file, using the code:

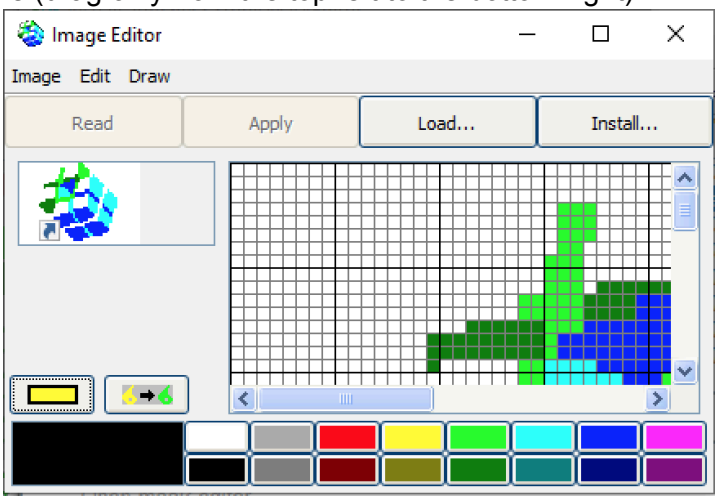
   ```
   Cormas installImageUserRequest
   ```

   Cormas guides you to select a file on your disk (.gif, .jpg, .png…), then to choose a class and a method name where to install this image.

2. or, by using the Capture option of the **Image Editor**: from the menu of the main VW interface = menu 'Painter' → Image Editor (or from the 'Tools' menu of the *GUI Painter Tool* → Tools → Image Editor)



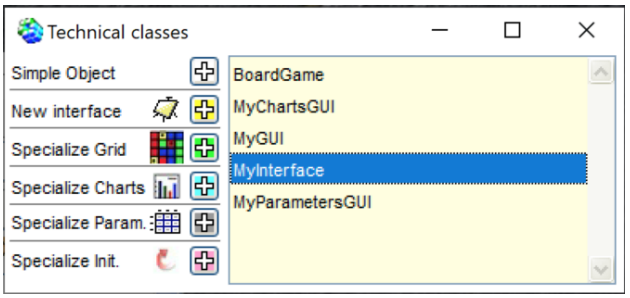
When the Image Editor is open, select Menu → Image → Capture. Then drag a part of your screen with a picture (drag only from the top-left to the bottom-right).



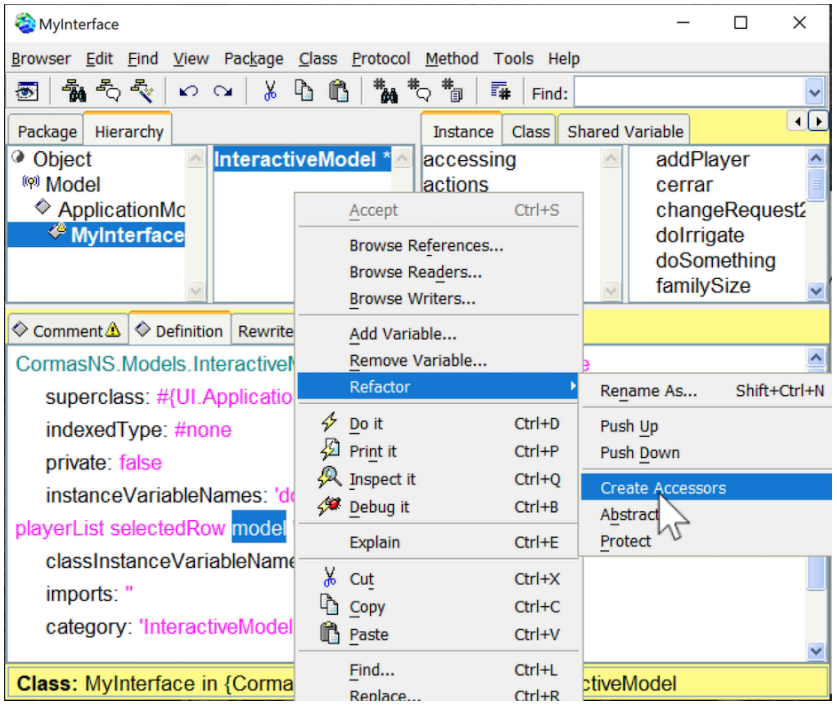
Then, install the new image as a method by clicking on the "**Install…**" button.

Adding the attribute 'model' to the GUI

It is common that an **Interface** should have an attribute called **'model'** that refers to your Cormas model. It is used for example to advise the model to continue the simulation when the interface will be closed by the user. So, we must add this attribute for your GUI. To do this, edit the class with a browser by double-clicking on `MyInterface` on the Technical classes toolbox:



and select the `'Definition'` tab. On the line `instanceVariableNames` (that corresponds to the list of attributes), write `model` (before the last quote): 'doIrrigate playerName role roleChoices familySize playerList selectedRow **model** ', then **Ctrl S** (or **Accept**). The new attribute `'model'` is created. To create the accessors, select `model` then right-click and select `Refactor → Create Accessors`:



This adds 2 public accessors for the attribute: `model` (the *getter* accessor) and `model:` (the *setter* accessor).

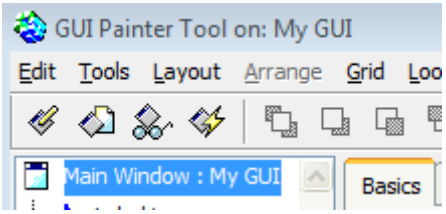
Setting GUI with the previous versions of Cormas

Cormas v. 2022.3 offers a set of tools for technical classes to facilitate the design and installation of new interfaces.
To open a new canvas (and the UI Painter), click on the 'new canvas' button of the main VW windows:  . It opens the 3 windows of the Painted tools as described in Creating a new interface.
After configuring your new interface, you need to install it and move it to your own model.

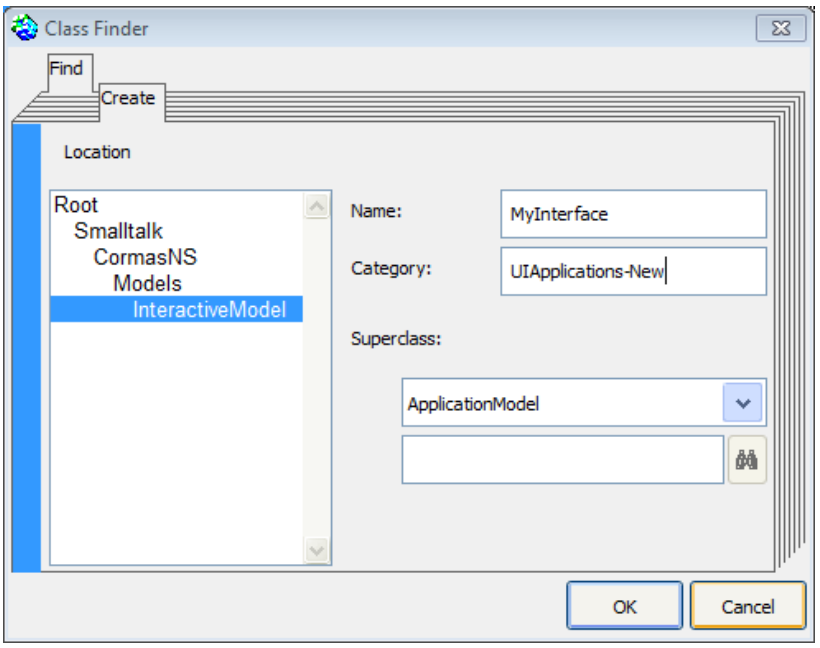
Install your GUI

Now that you have a minimalist interface, let's save it, that is to say to "install" it in your model. This writes the canvas specification into a "spec" method. This specification will be used when opening the window (you can also edit the specification later by reopening it in the UI Painter).

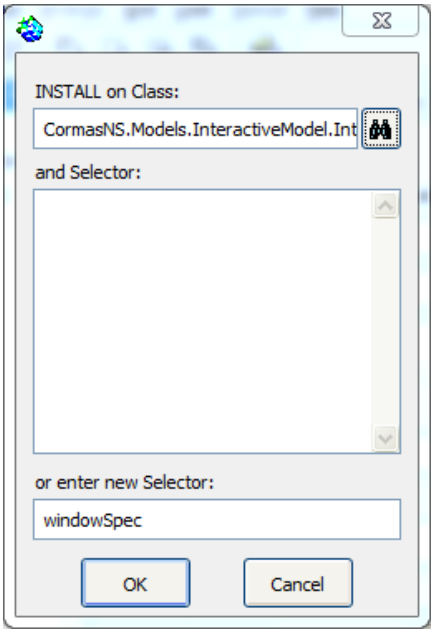
To install your GUI, click on the first button of the GUI Painter Tool: 



In order to define where the specification must be stored, a class and a method must be selected. For that, click on the button  then navigate to your model by clicking successively on "CormasNS" → "Models" → YourModel (Ex: InteractiveModel). If no interface has been defined yet, click on the 'Create' tab:



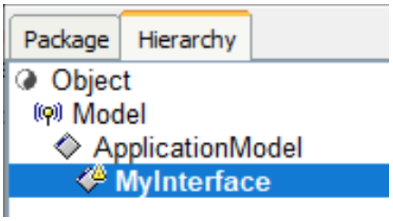
Enter the name of your interface (ex: MyInterface).
Then set the name of the method where the specification of the GUI is stored (by default it is *windowSpec*):



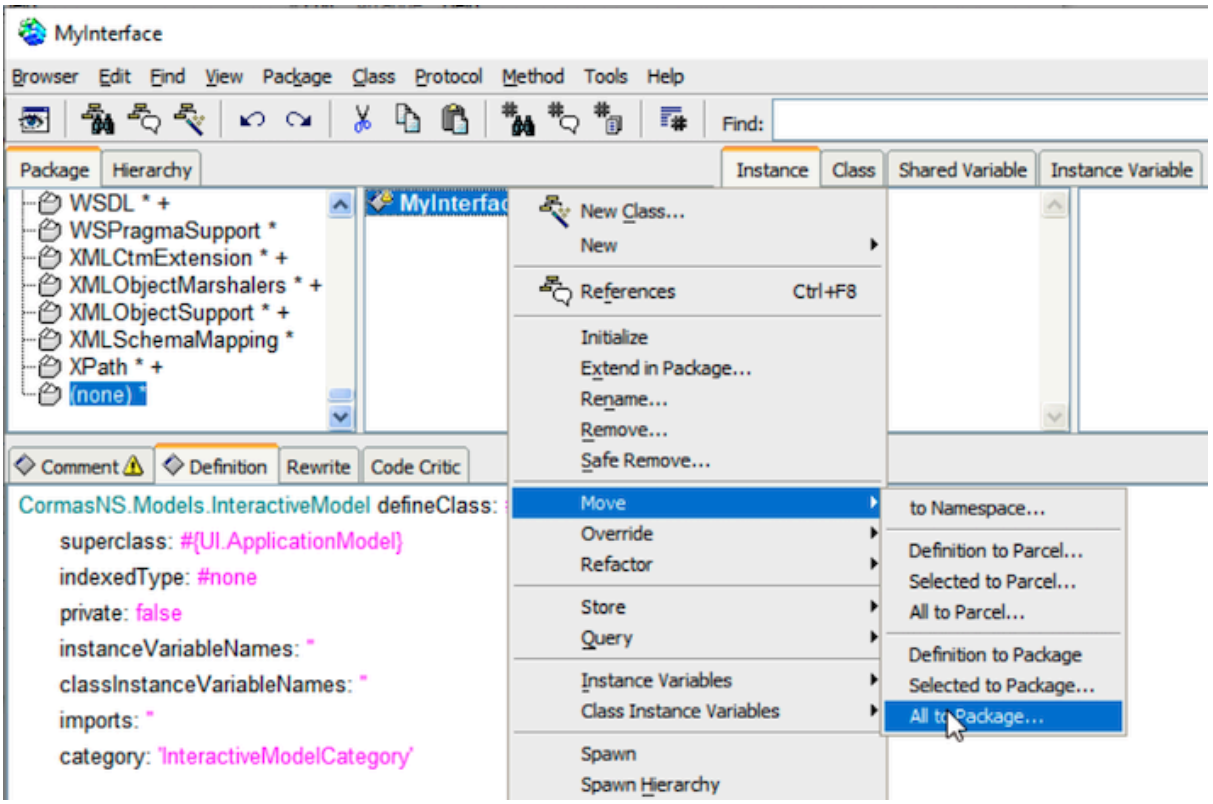
Each time a new widget will be added on the interface, the 'install' procedure must be performed to save those changes.

Move the interface to your model

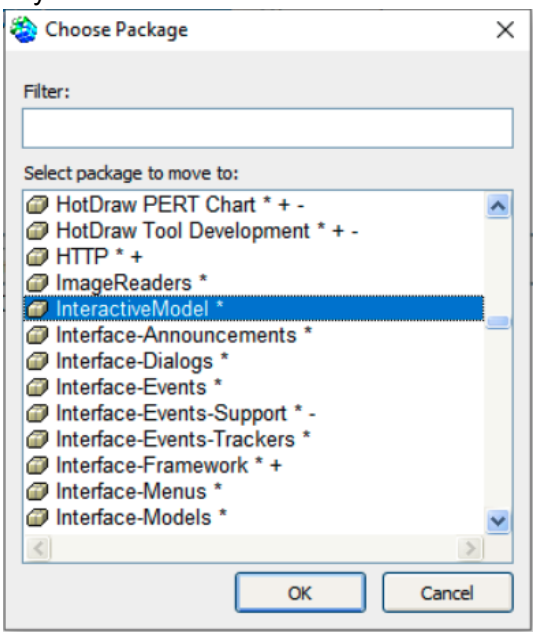
If you have created your interface from a blank canvas, then it is necessary to associate this class with your model. In other words, you need to add this interface to your model's package and namespace (only once): at the beginning, your new class is not packaged.
To do this, open a class browser (for example by double clicking on a class of your model):



Then select the "Package" tab. Navigate to the bottom and select the package **(none) ***.
To move your interface, right click on the class name and from the `Menu → Move → All to Package...`

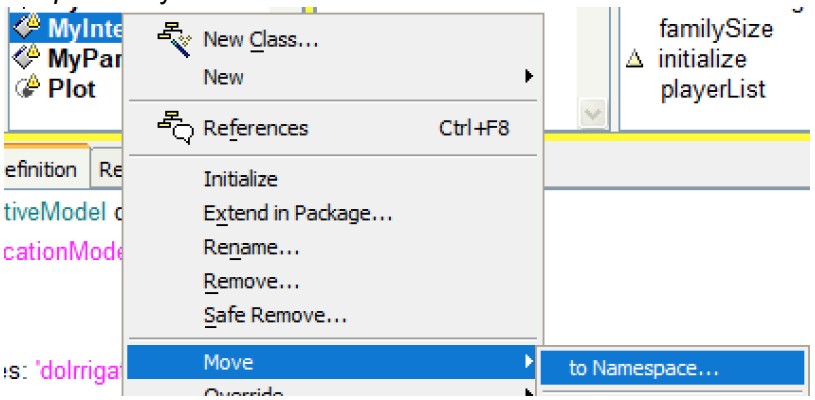


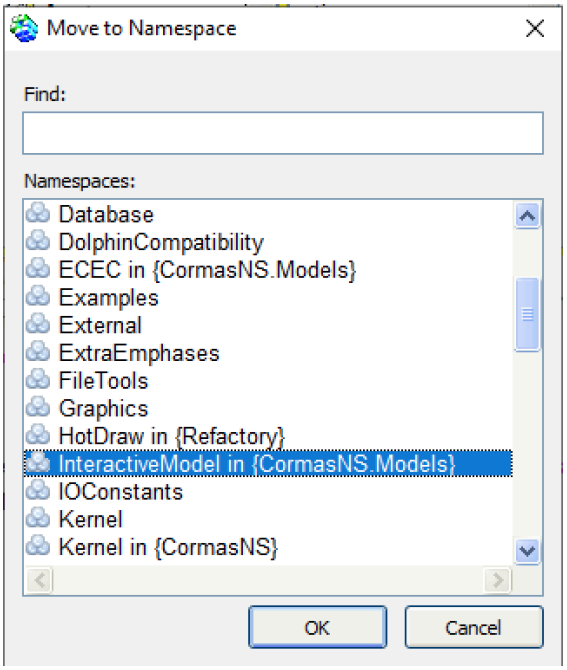
Thus, select the package of your model:



This move of the interface to your model's package must be done only once. From now on, your interface belongs to your model and will be saved each time the model is saved.

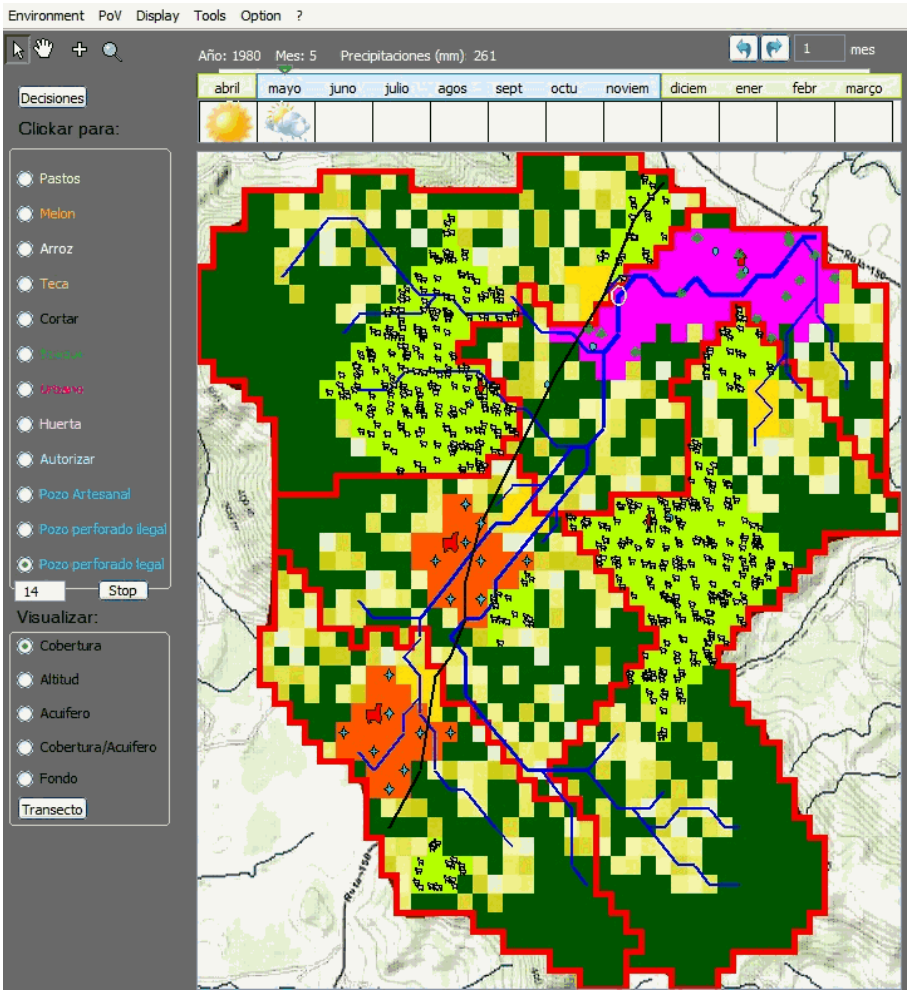
Finally, in order to facilitate the short access to the other classes of your model, move your class to the *Namespace* of your model:



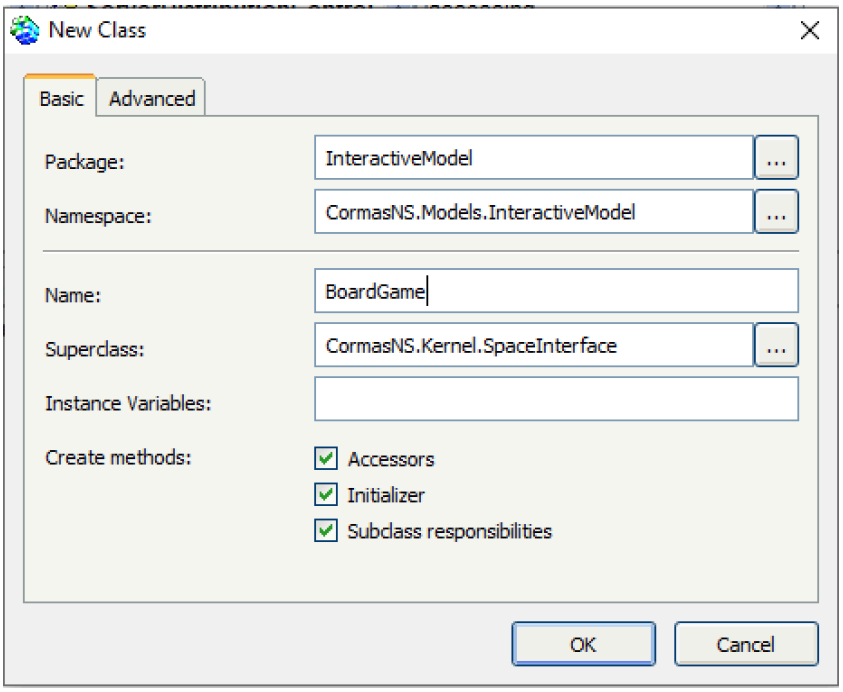


Spatial grid as Game board

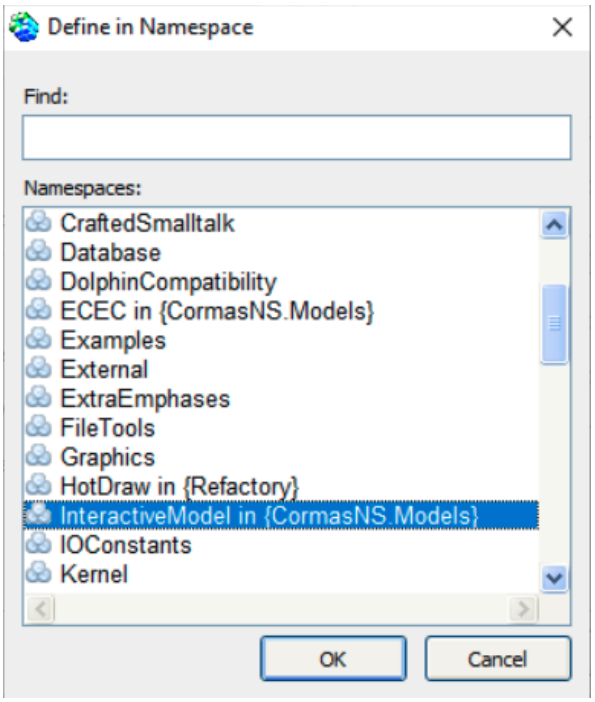
Cormas offers a default spatial interface. It is possible to modify this grid to adapt it to your needs. It is open from the menu Visualisation → Spatial grid.
Example of game board in *ContamiCuenca* model:



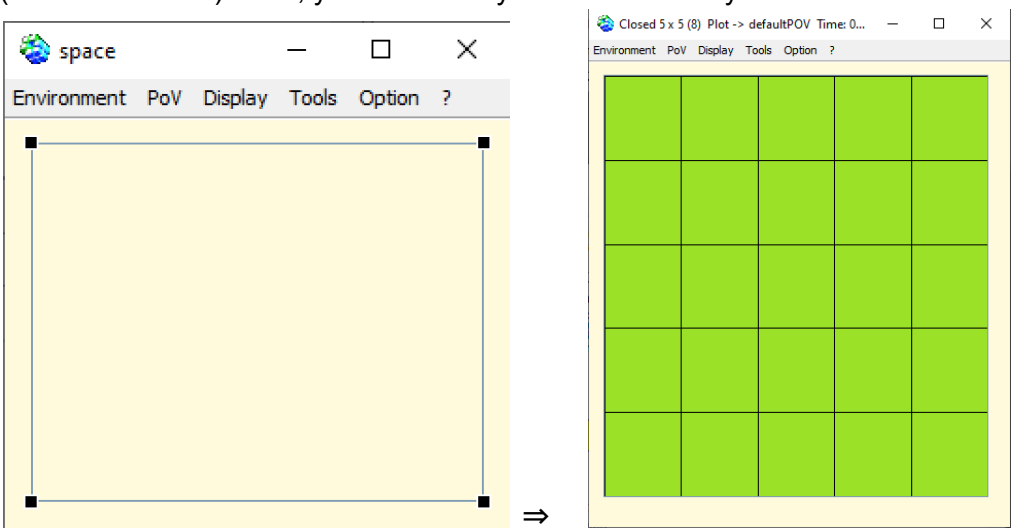
To do this, look for the **SpatialInterface** class. Then with the right button, select '*New Class…*'. Define the Package and the Namespace corresponding to your model. Give a name to this new class (ex: *BoardGame*):



For example, to set the nameSpace, click on  , then select the nameSpace of your model:



When it's done, drag and drop the `windowSpec` method from SpaceInterface to your class (ex: BoardGame). Now, you can modify BoardGame as you want:



In the main class of your model (e.g. *InteractiveModel*), redefine the method `spaceInterfaceType` as follows:

```
spaceInterfaceType
        ^ BoardGame
```

Adapting the charts

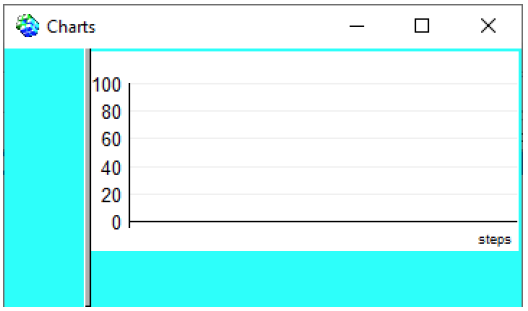
Cormas offers a default charts' interface. As previously, it is possible to modify this interface to adapt it to your needs. It is open via the Cormas menu `'Visualisation' → Charts`.

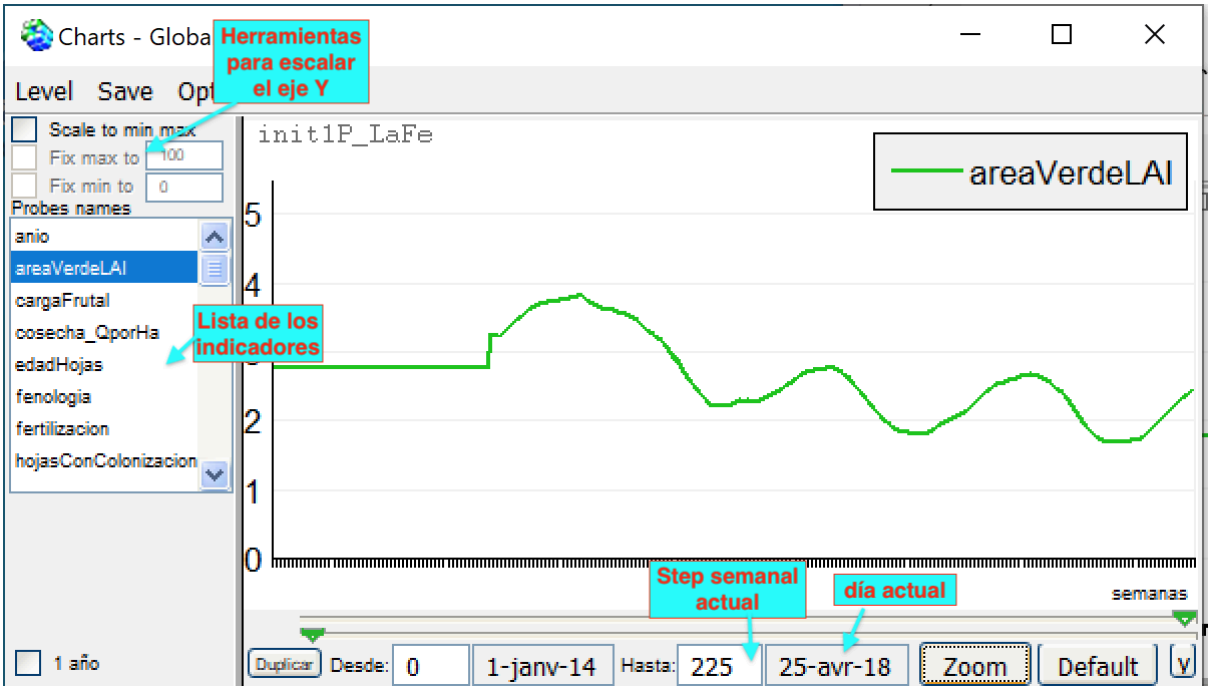
To do this, look for the **ChartsInterface** class. Then with the right button, select '*New Class…*'. Define the Package and the Namespace corresponding to your model. Give a name to this new class (ex: MyChartsGUI).

In the main class of your model (e.g. InteractiveModel), redefine the method `ChartsInterfaceType` as follows:

```
chartsInterfaceType
        ^ MyChartsGUI
```

Now, to modify the charts interface, drag and drop the `windowSpec` method from the **ChartsInterface** class to your own class.
Example of charts' interface for InteractiveModel:





In this specialization of the **ChartsInterface**, a CheckBox ("1 año") has been added at the bottom-left. It shows the curves only for the last year. In addition to the standard steps (integers), the time is presented in the form of dates. Finally, a button ("y") has been added at the bottom right to show the curves truncated in one-year sections.

Adapting the parameters' interface

Cormas offers a default parameters interface. As previously, it is possible to modify this interface to adapt it to your needs. It will open by clicking on the 'Parameters' button .
To do this, look for the **ParameterInterface** class. Then with the right button, select '*New Class…*'. Define the Package and the Namespace corresponding to your model. Give a name to this new class (ex: MyChartsGUI).

In the main class of your model (e.g. InteractiveModel), redefine the method `ParameterInterfaceType` as follows:

```
parameterInterfaceType
        ^ MyParametersGUI
```

Now, to modify the parameters interface, drag and drop the `windowSpec` method from the **ParameterInterface** class to your own class.
Example of parameters' interface for InteractiveModel:

